



1. 操作系统的运行环境

操作系统运行环境

os引导

os内核

CPU的双重工作模式

特权指令

系统调用

中断和异常处理



1. 操作系统的运行环境

OS引导

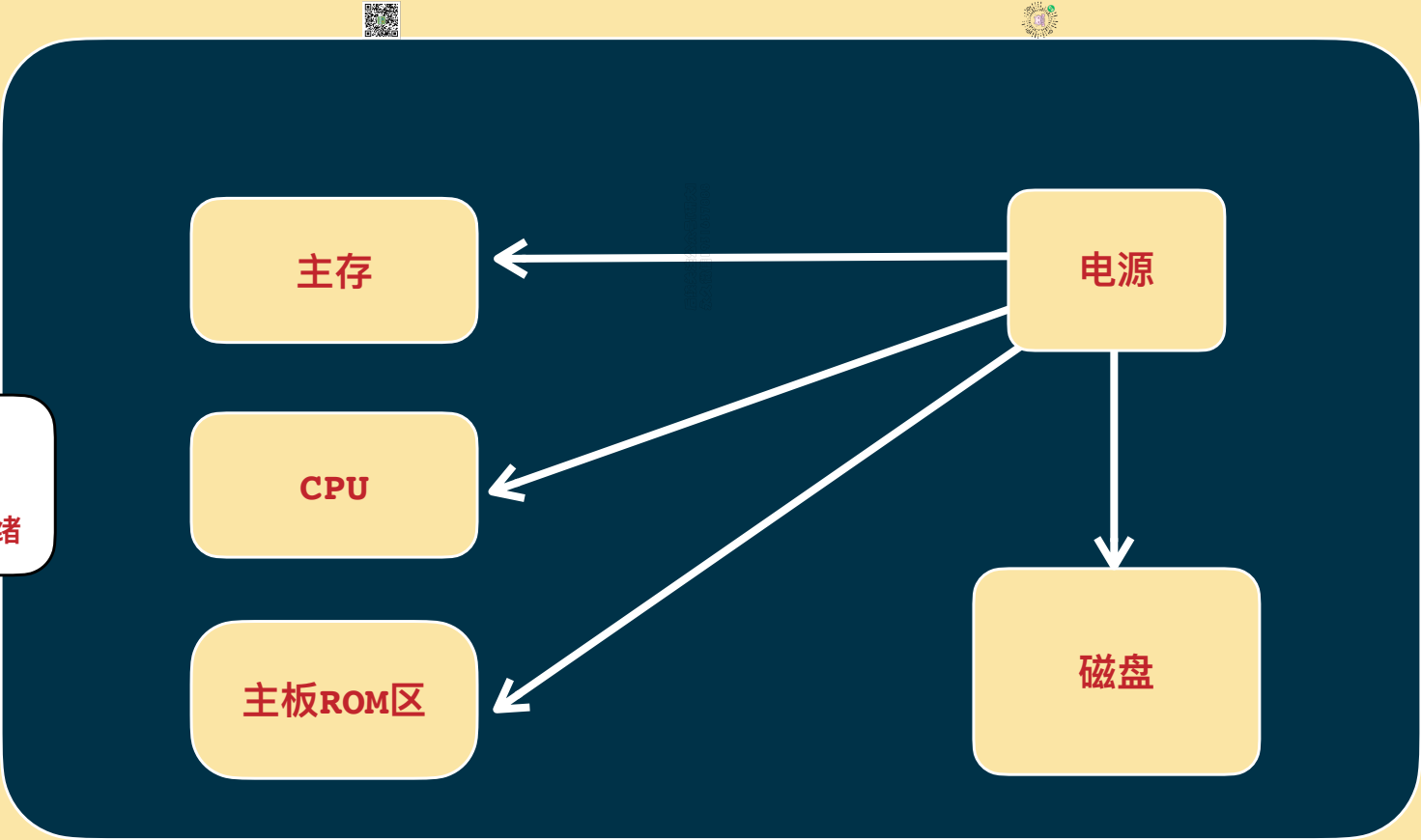
OS内核

CPU的双重工作模式

特权指令

系统调用

中断和异常处理

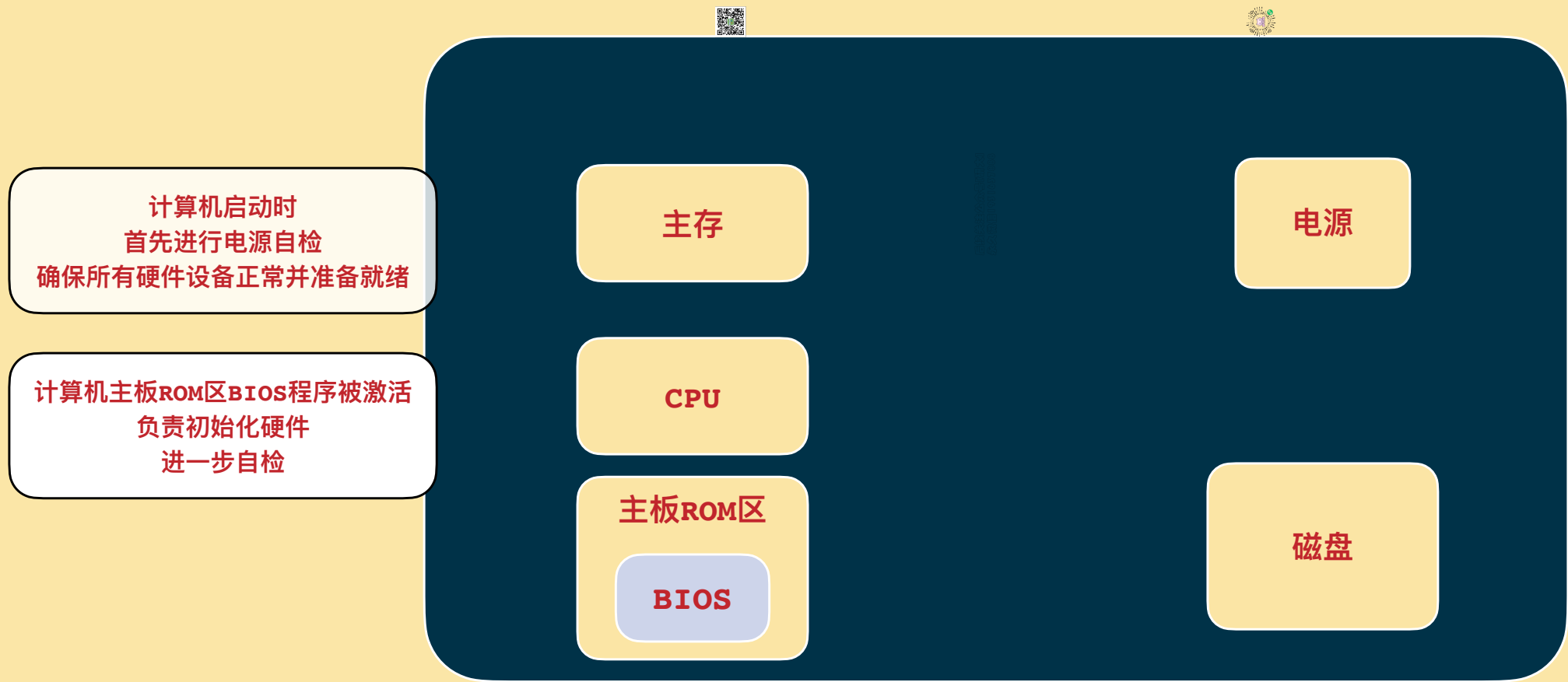


计算机启动时
首先进行电源自检
确保所有硬件设备正常并准备就绪



1. 操作系统的运行环境

- os引导
- os内核
- CPU的双重工作模式
- 特权指令
- 系统调用
- 中断和异常处理

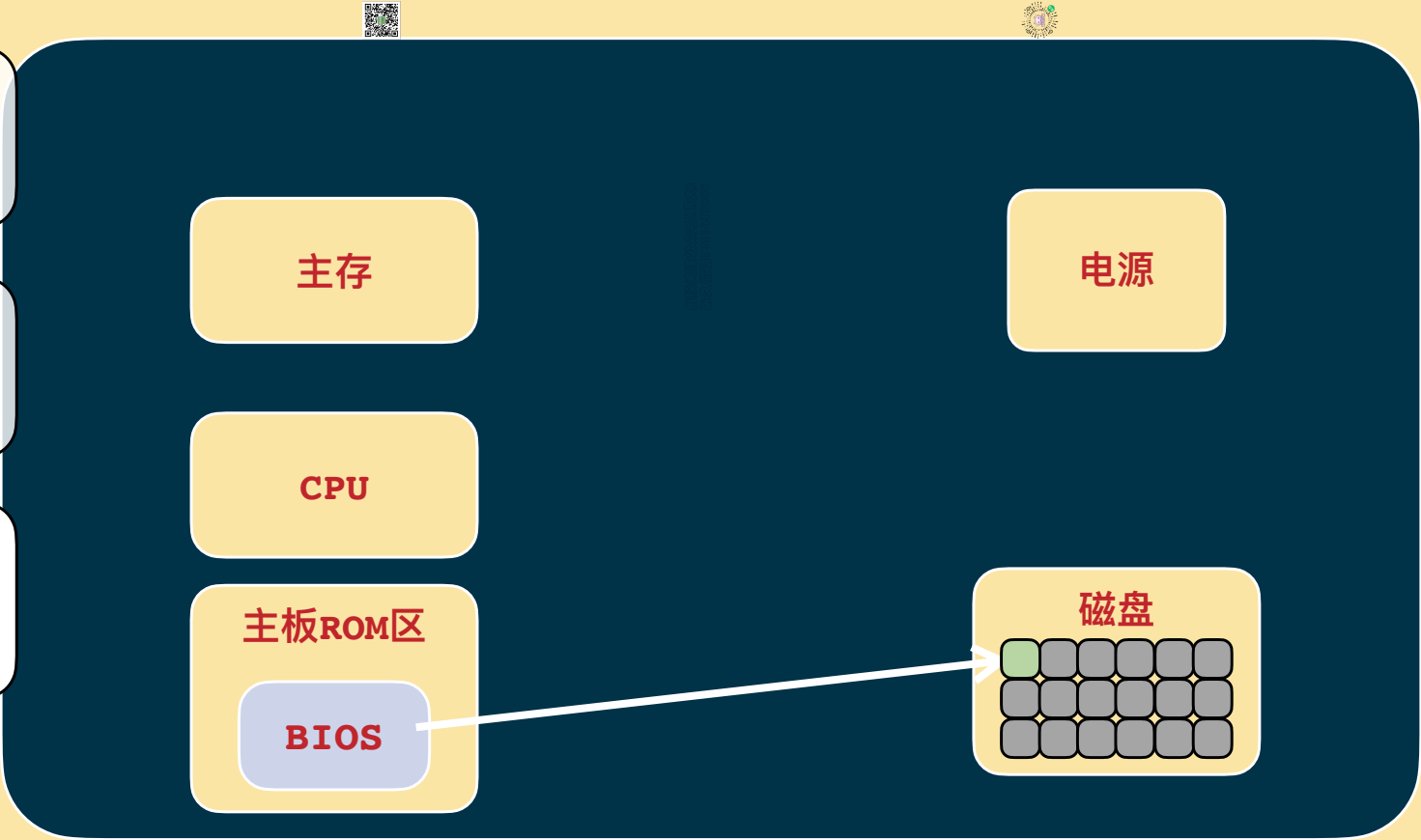




1. 操作系统的运行环境

- os引导
- os内核
- CPU的双重工作模式
- 特权指令
- 系统调用
- 中断和异常处

- 计算机启动时
首先进行电源自检
确保所有硬件设备正常并准备就绪
- 计算机主板ROM区BIOS程序被激活
负责初始化硬件
进一步自检
- BIOS读取硬盘0号扇区的主引导记录
MBR
MBR包括磁盘引导程序和分区表信息





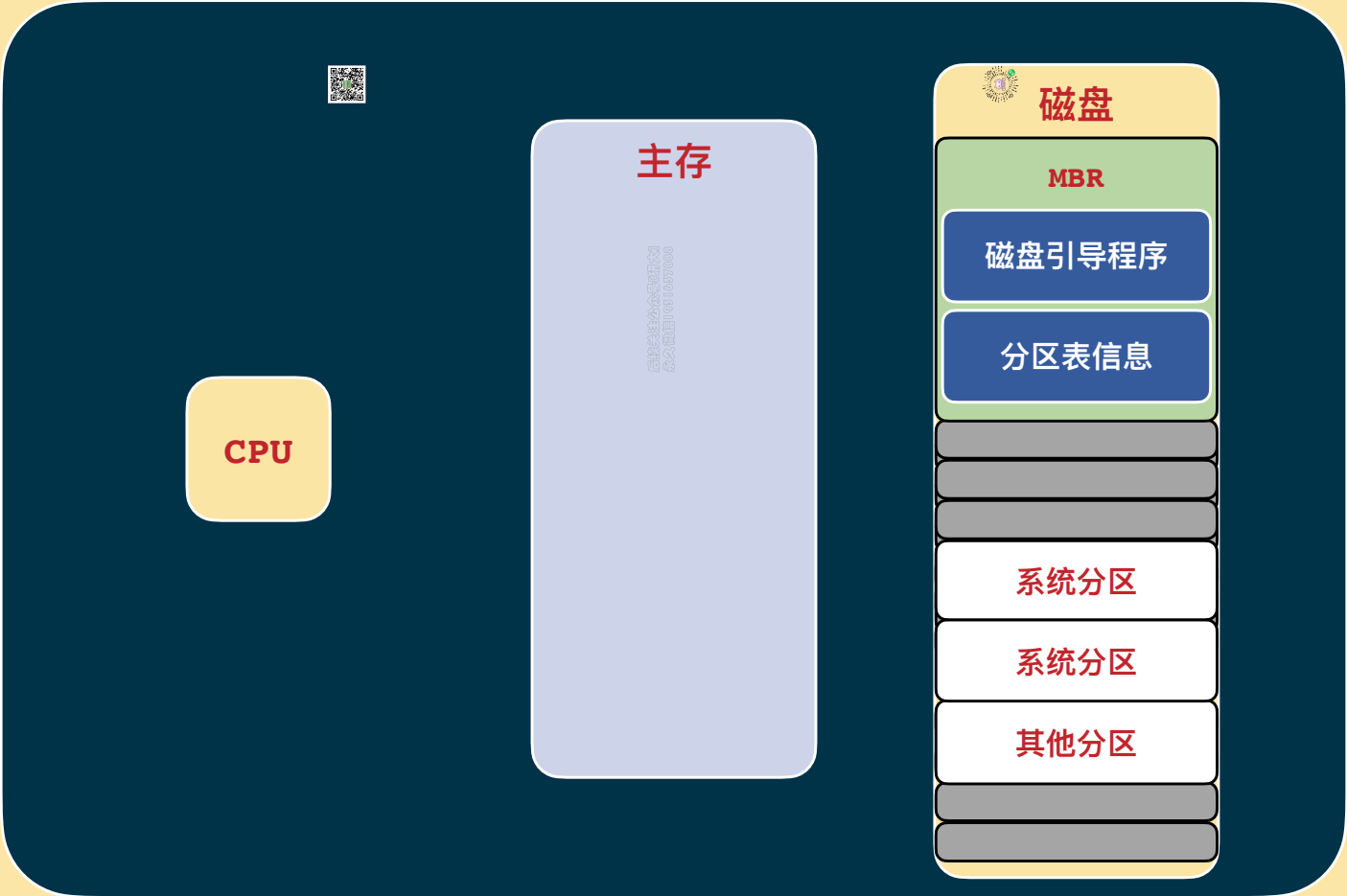
1. 操作系统的运行环境

- os引导
- os内核
- CPU的双重工作模式
- 特权指令
- 系统调用
- 中断和异常处

计算机启动时
首先进行电源自检
确保所有硬件设备正常并准备就绪

计算机主板ROM区BIOS程序被激活
负责初始化硬件
进一步自检

BIOS读取硬盘0号扇区的主引导记录
MBR
MBR包括磁盘引导程序和分区表信息

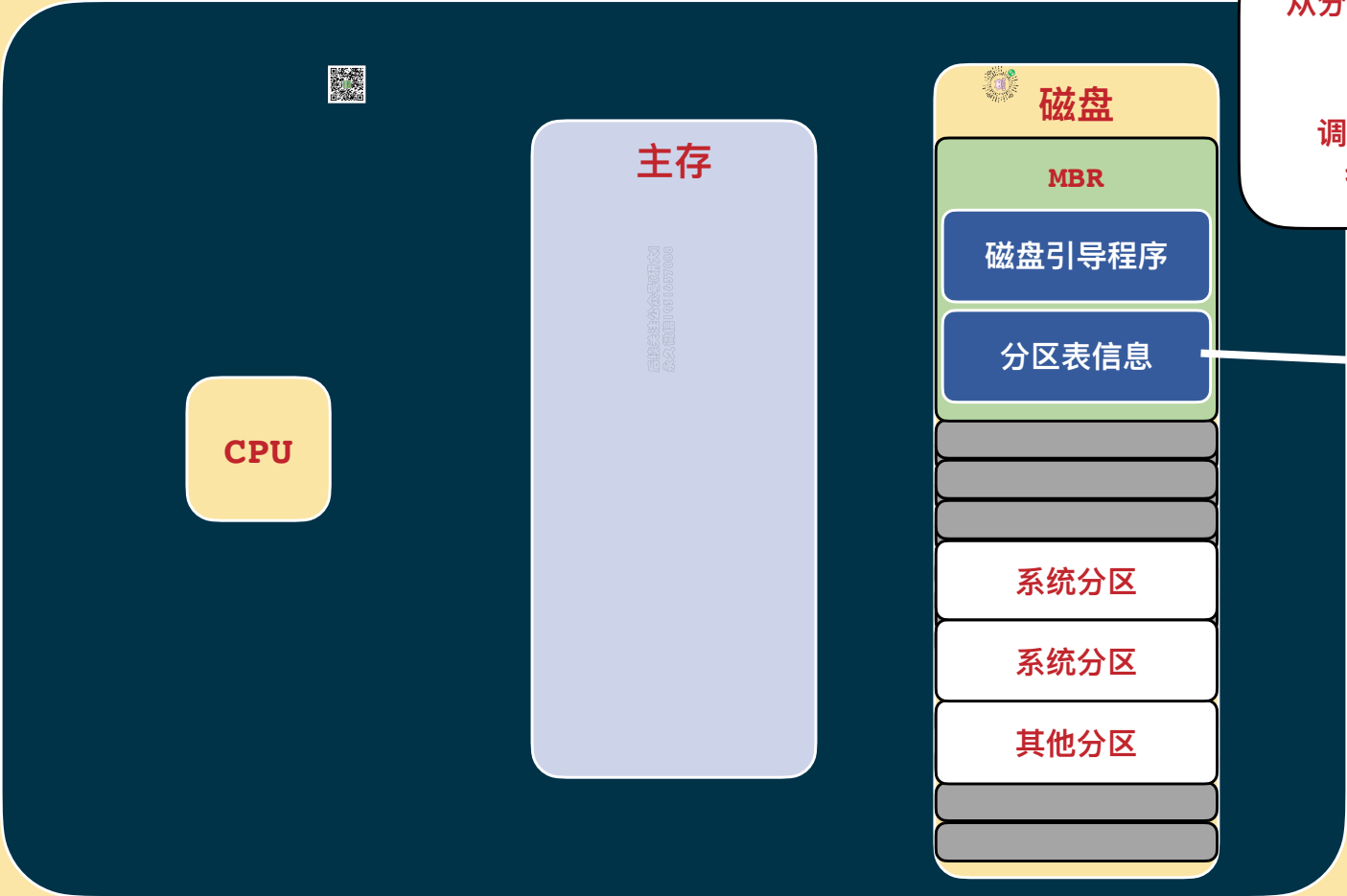




1. 操作系统的运行环境

- os引导
- os内核
- CPU的双重工作模式
- 特权指令
- 系统调用
- 中断和异常处

- 计算机启动时
首先进行电源自检
确保所有硬件设备正常并准备就绪
- 计算机主板ROM区BIOS程序被激活
负责初始化硬件
进一步自检
- BIOS读取硬盘0号扇区的主引导记录
MBR
MBR包括磁盘引导程序和分区表信息



从分区表可得知操作系统所在系统分区位置
系统分区的0号扇区
存放分区引导记录PBR
调入该扇区内的操作引导程序进入主存
得知os内核位置并将其加载入主存

(分区标识) 盘符 起始地址

装有windows	c	789
装有linux	d	1788
	e	2981

有多系统时
想要启动哪个系统就加载哪个分区
若需要启动windows
则将c盘第一个扇区调入内存
该扇区也被称为分区引导记录PBR



1. 操作系统的运行环境

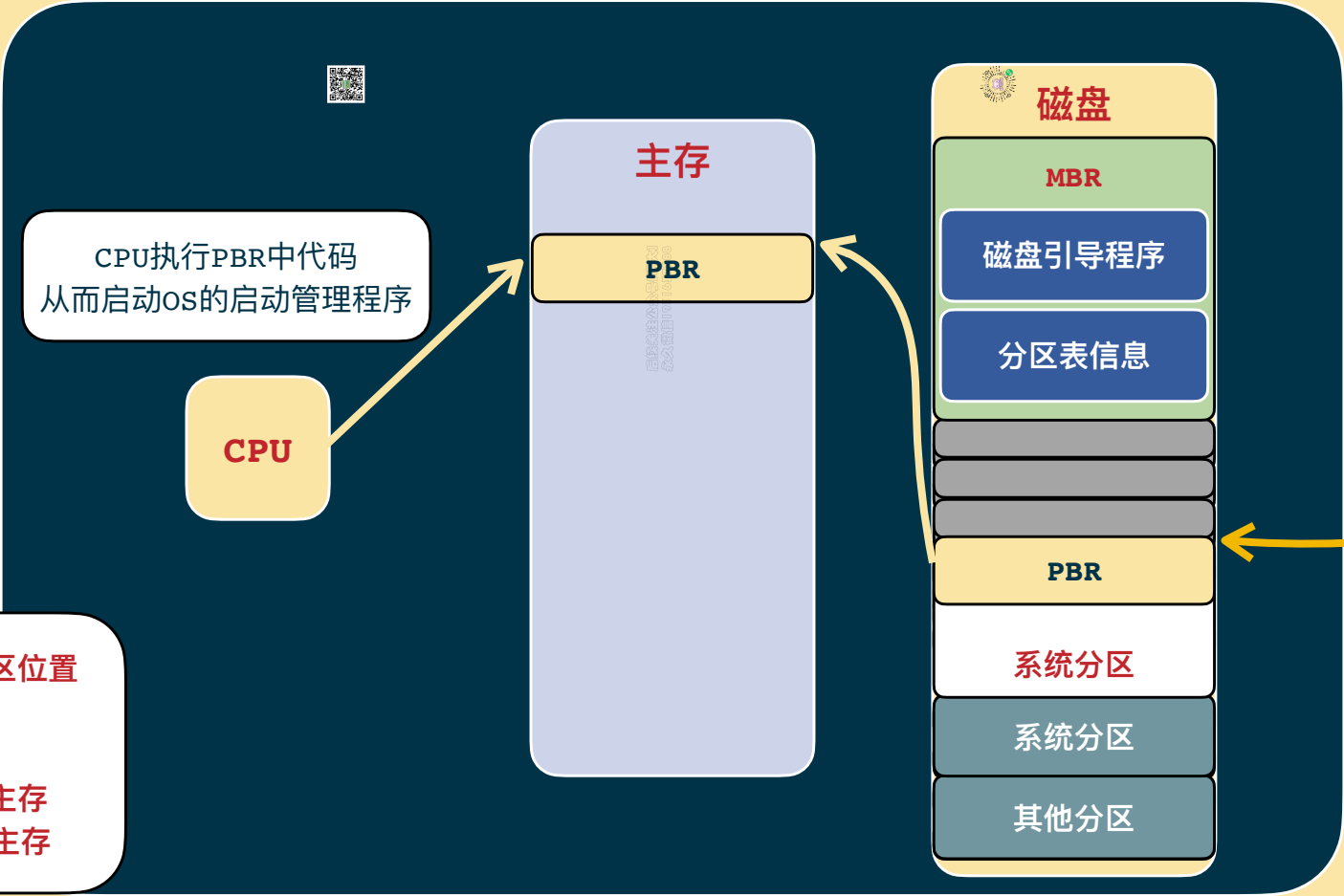
- os引导
- os内核
- CPU的双重工作模式
- 特权指令
- 系统调用
- 中断和异常处

计算机启动时
首先进行电源自检
确保所有硬件设备正常并准备就绪

计算机主板ROM区BIOS程序被激活
负责初始化硬件
进一步自检

BIOS读取硬盘0号扇区的主引导记录
MBR
MBR包括磁盘引导程序和分区表信息

从分区表可得知操作系统所在系统分区位置
系统分区的0号扇区
存放分区引导记录PBR
调入该扇区内的启动管理程序进入主存
之后得知os内核位置并将其加载入主存



(分区标识) 盘符 起始地址

装有windows	c	789
装有linux	d	1788
	e	2981

有多系统时
想要启动哪个系统就加载哪个分区
若需要启动windows
则将c盘第一个扇区调入内存
该扇区也被称为分区引导记录PBR



1. 操作系统的运行环境

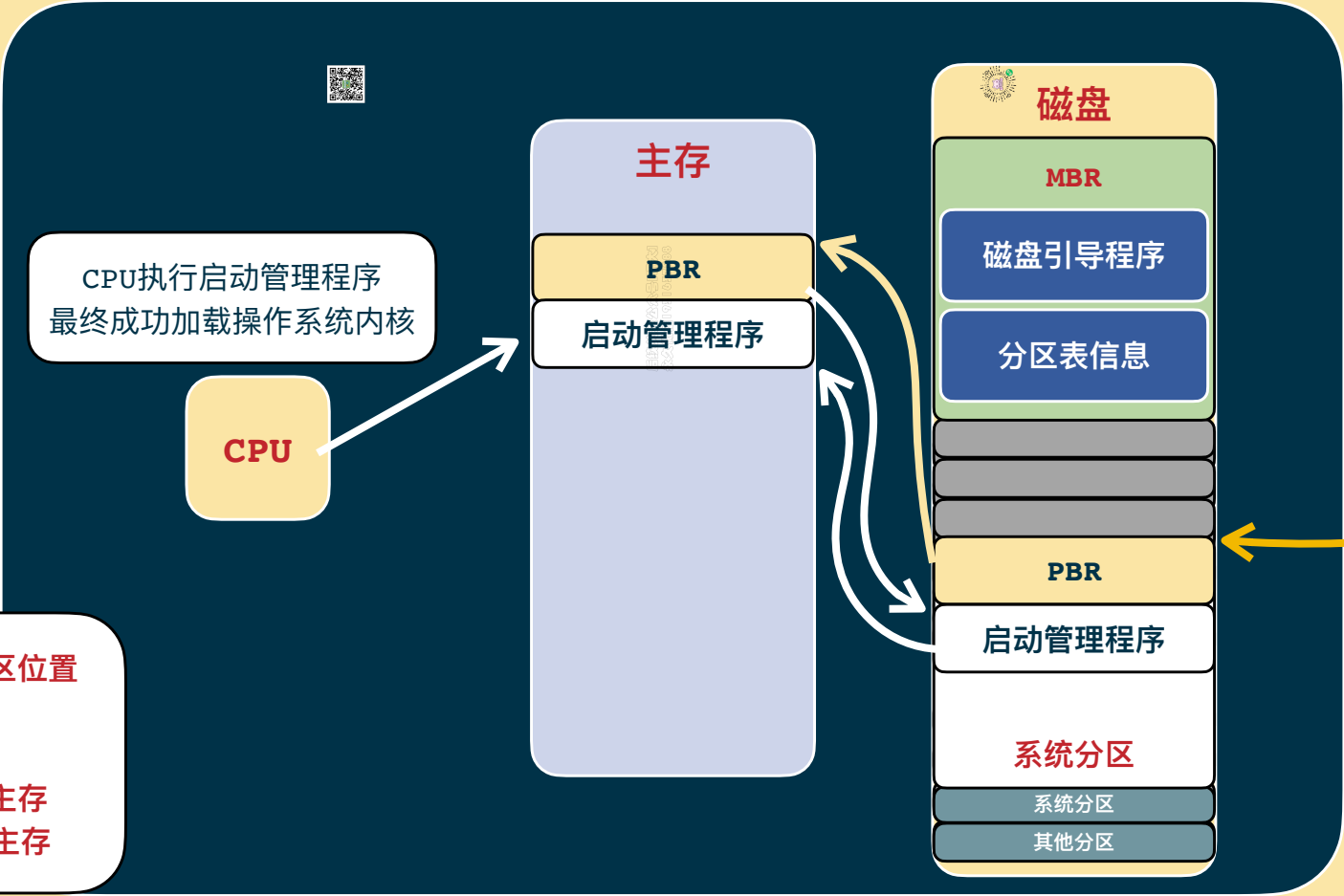
- os引导
- os内核
- CPU的双重工作模式
- 特权指令
- 系统调用
- 中断和异常处

计算机启动时
首先进行电源自检
确保所有硬件设备正常并准备就绪

计算机主板ROM区BIOS程序被激活
负责初始化硬件
进一步自检

BIOS读取硬盘0号扇区的主引导记录
MBR
MBR包括磁盘引导程序和分区表信息

从分区表可得知操作系统所在系统分区位置
系统分区的0号扇区
存放分区引导记录PBR
调入该扇区内的启动管理程序进入主存
之后得知os内核位置并将其加载入主存



(分区标识) 盘符 起始地址

装有windows	c	789
装有linux	d	1788
	e	2981

有多系统时
想要启动哪个系统就加载哪个分区
若需要启动windows
则将c盘第一个扇区调入内存
该扇区也被称为分区引导记录PBR



1. 操作系统的运行环境

os引导

os内核

CPU的双重工作模式

特权指令

系统调用

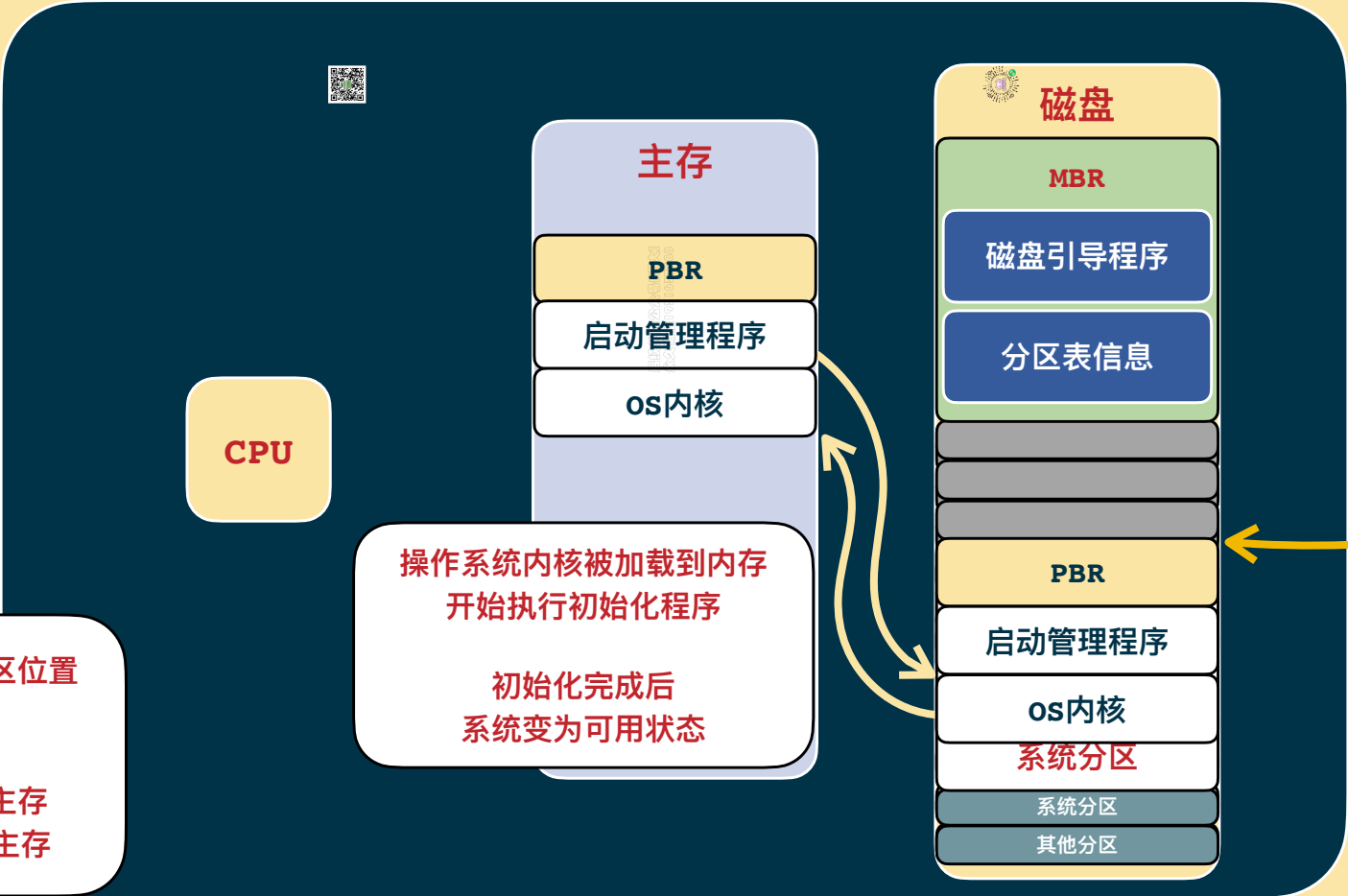
中断和异常处

计算机启动时
首先进行电源自检
确保所有硬件设备正常并准备就绪

计算机主板ROM区BIOS程序被激活
负责初始化硬件
进一步自检

BIOS读取硬盘0号扇区的主引导记录
MBR
MBR包括磁盘引导程序和分区表信息

从分区表可得知操作系统所在系统分区位置
系统分区的0号扇区
存放分区引导记录PBR
调入该扇区内的启动管理程序进入主存
之后得知os内核位置并将其加载入主存



(分区标识) 盘符 起始地址

装有windows	c	789
装有linux	d	1788
	e	2981

有多系统时
想要启动哪个系统就加载哪个分区
若需要启动windows
则将c盘第一个扇区调入内存
该扇区也被称为分区引导记录PBR



1. 操作系统的运行环境

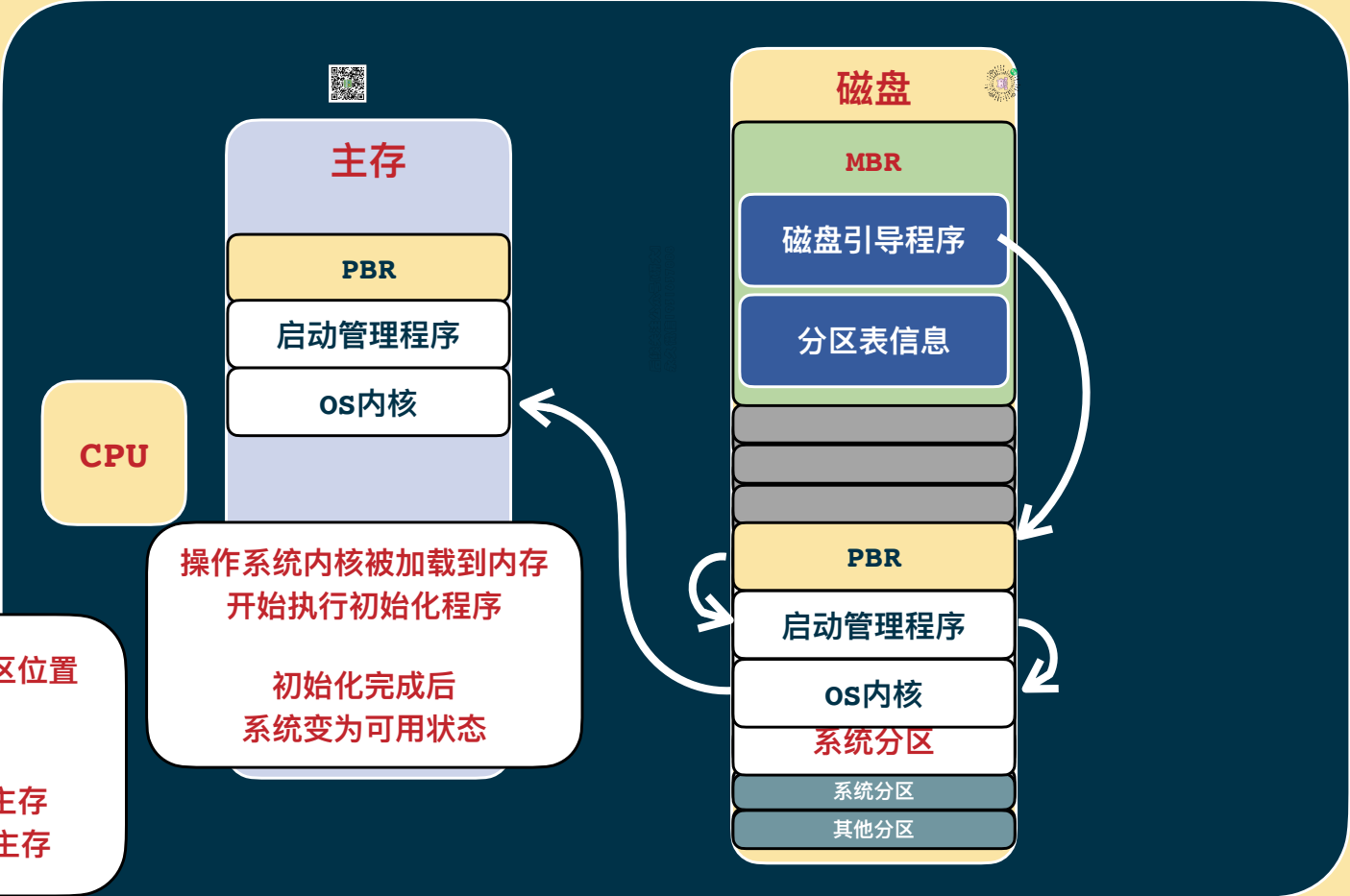
- os引导
- os内核
- CPU的双重工作模式
- 特权指令
- 系统调用
- 中断和异常处

计算机启动时
首先进行电源自检
确保所有硬件设备正常并准备就绪

计算机主板ROM区BIOS程序被激活
负责初始化硬件
进一步自检

BIOS读取硬盘0号扇区的主引导记录
MBR
MBR包括磁盘引导程序和分区表信息

从分区表可得知操作系统所在系统分区位置
系统分区的0号扇区
存放分区引导记录PBR
调入该扇区内的启动管理程序进入主存
之后得知os内核位置并将其加载入主存



(分区标识) 盘符 起始地址

装有windows	c	789
装有linux	d	1788
	e	2981

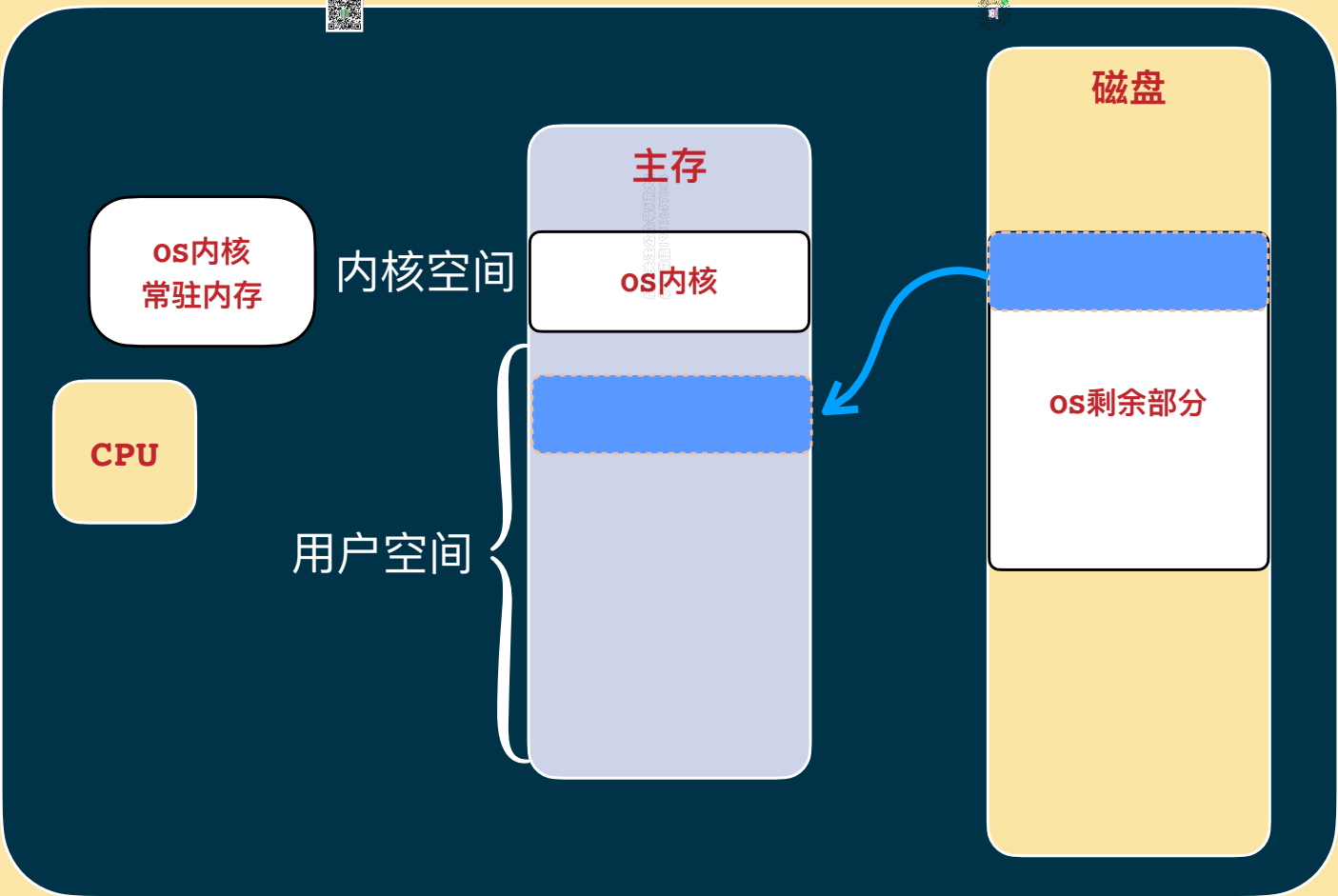
有多系统时
想要启动哪个系统就加载哪个分区
若需要启动windows
则将c盘第一个扇区调入内存
该扇区也被称为分区引导记录PBR



1. 操作系统的运行环境

- os引导
- os内核**
- CPU的双重工作模式
- 特权指令
- 系统调用
- 中断和异常处理

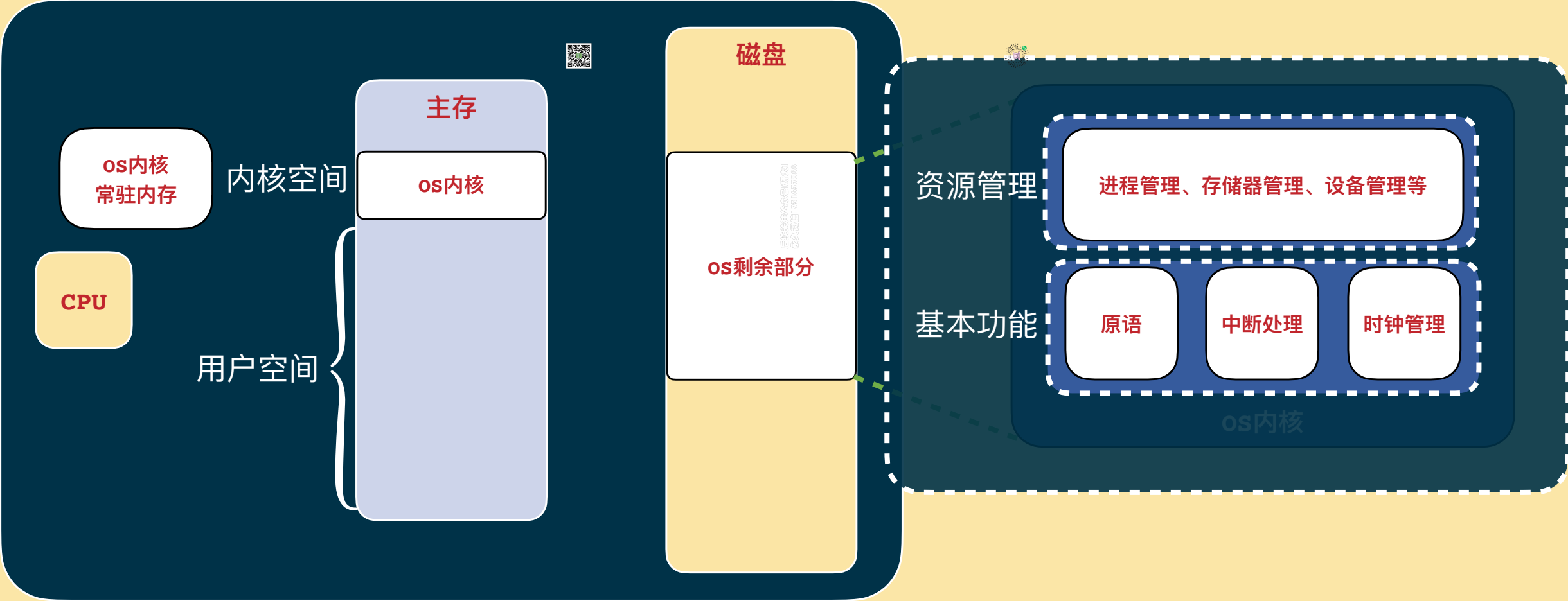
os中与硬件紧密相关的模块，负责管理系统的各种资源





1. 操作系统的运行环境

- os引导
- os内核**
- CPU的双重工作模式
- 特权指令
- 系统调用
- 中断和异常处理

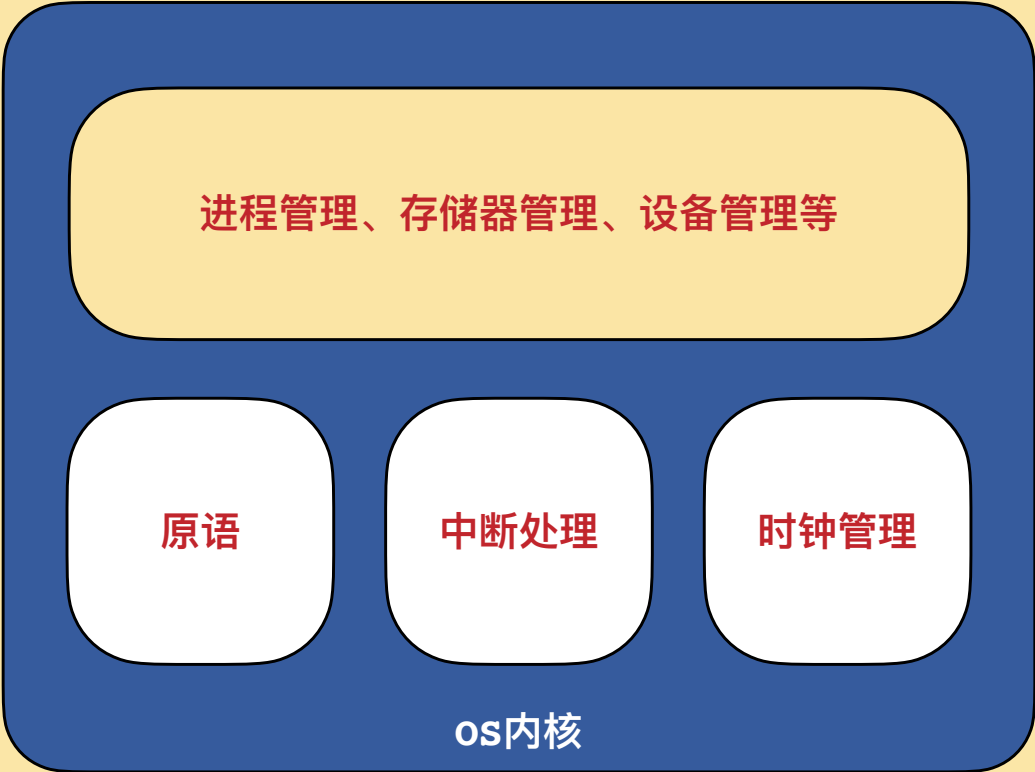




1. 操作系统的运行环境

- os引导
- os内核**
- CPU的双重工作模式
- 特权指令
- 系统调用
- 中断和异常处理

os中与硬件紧密相关的模块，负责管理系统的各种资源



进程管理

确保os可以高效、公平、安全地同时运行多个应用程序

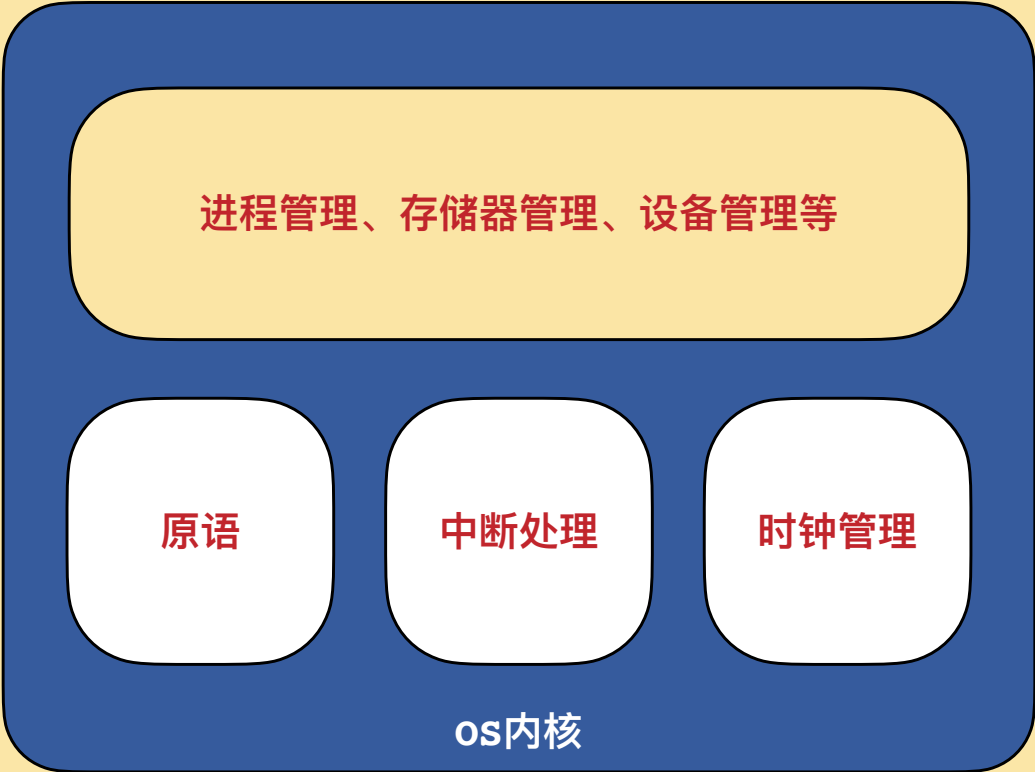
第二章将花费巨大篇幅学习



1. 操作系统的运行环境

- os引导
- os内核**
- CPU的双重工作模式
- 特权指令
- 系统调用
- 中断和异常处理

os中与硬件紧密相关的模块，负责管理系统的各种资源



存储器管理

确保有效、安全地分配管理计算机系统中的内存资源

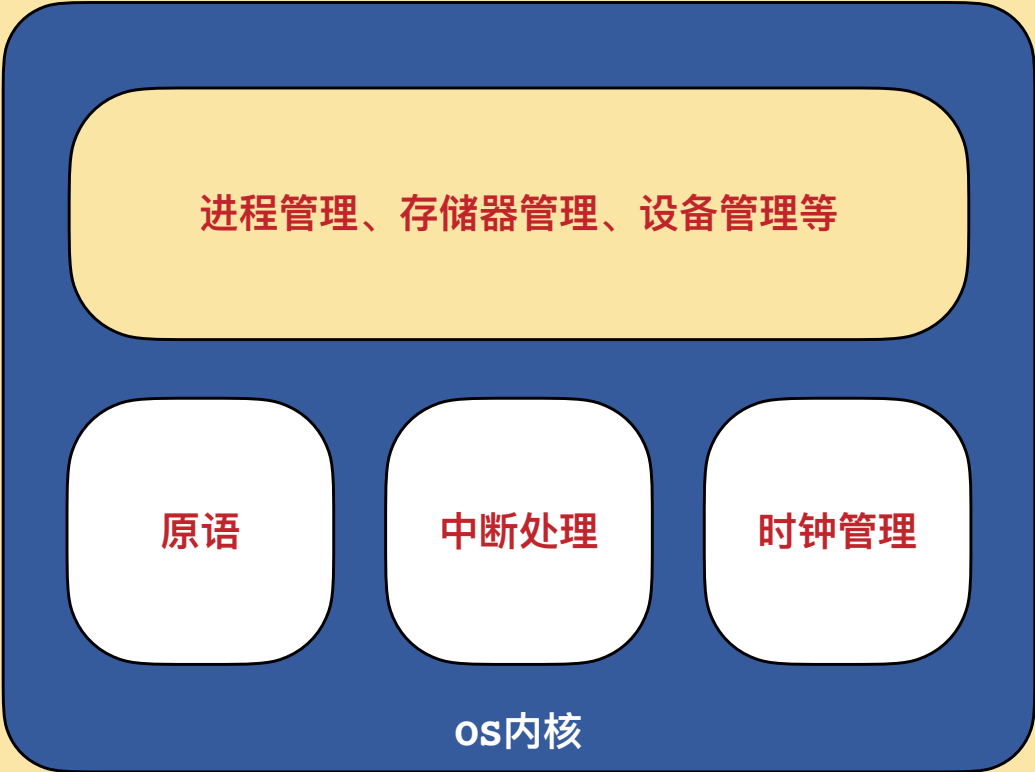
第三章将花费大篇幅学习



1. 操作系统的运行环境

- os引导
- os内核**
- CPU的双重工作模式
- 特权指令
- 系统调用
- 中断和异常处理

os中与硬件紧密相关的模块，负责管理系统的各种资源



设备管理

设备管理

确保所有外围设备和硬件都能有效且安全地与计算机系统交互

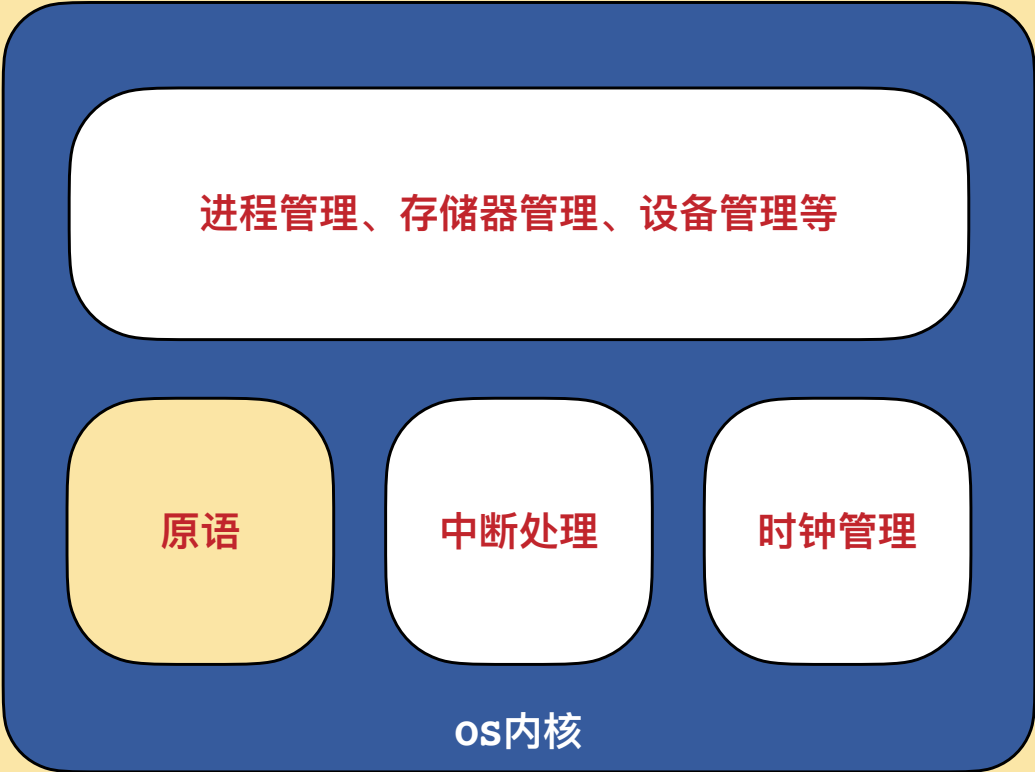
第五章将进一步学习



1. 操作系统的运行环境

- os引导
- os内核**
- CPU的双重工作模式
- 特权指令
- 系统调用
- 中断和异常处理

os中与硬件紧密相关的模块，负责管理系统的各种资源



原语

在单个步骤中完成os的基础构件
处于操作系统最低层，运行时不可以被打断
运行时长较短，调用频繁

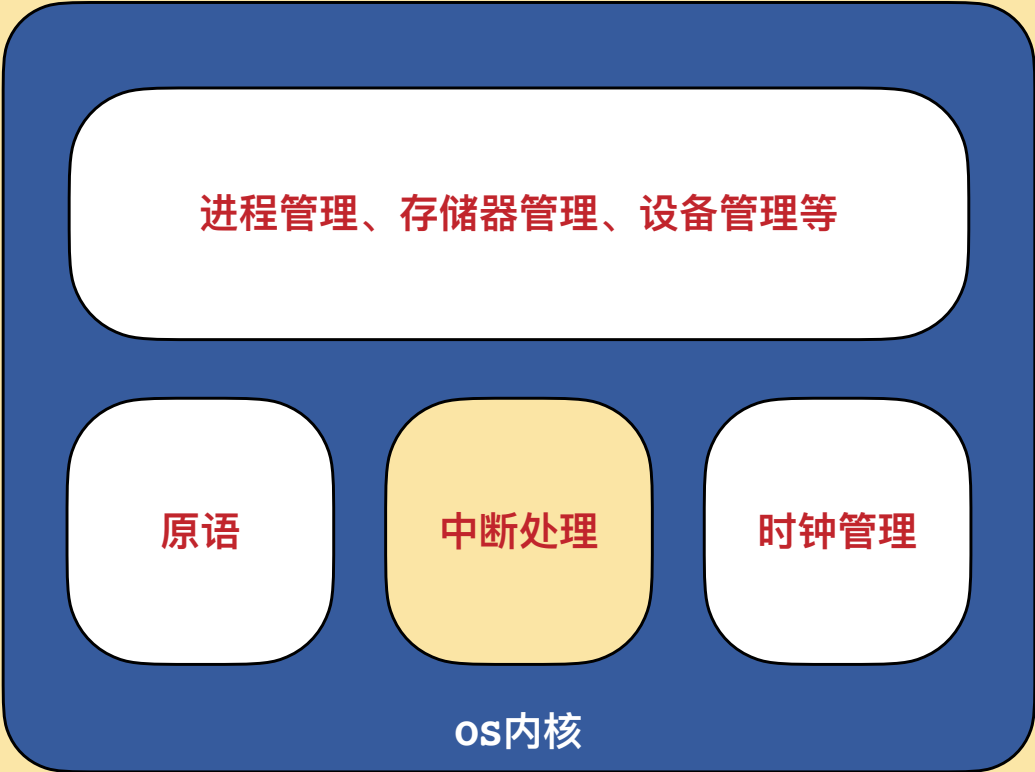
linux中的常见原语 {
fork()
exec()
wait()
open()
...



1. 操作系统的运行环境

- os引导
- os内核**
- cpu的双重工作模式
- 特权指令
- 系统调用
- 中断和异常处理

os中与硬件紧密相关的模块，负责管理系统的各种资源



进程管理、存储器管理、设备管理等

中断处理

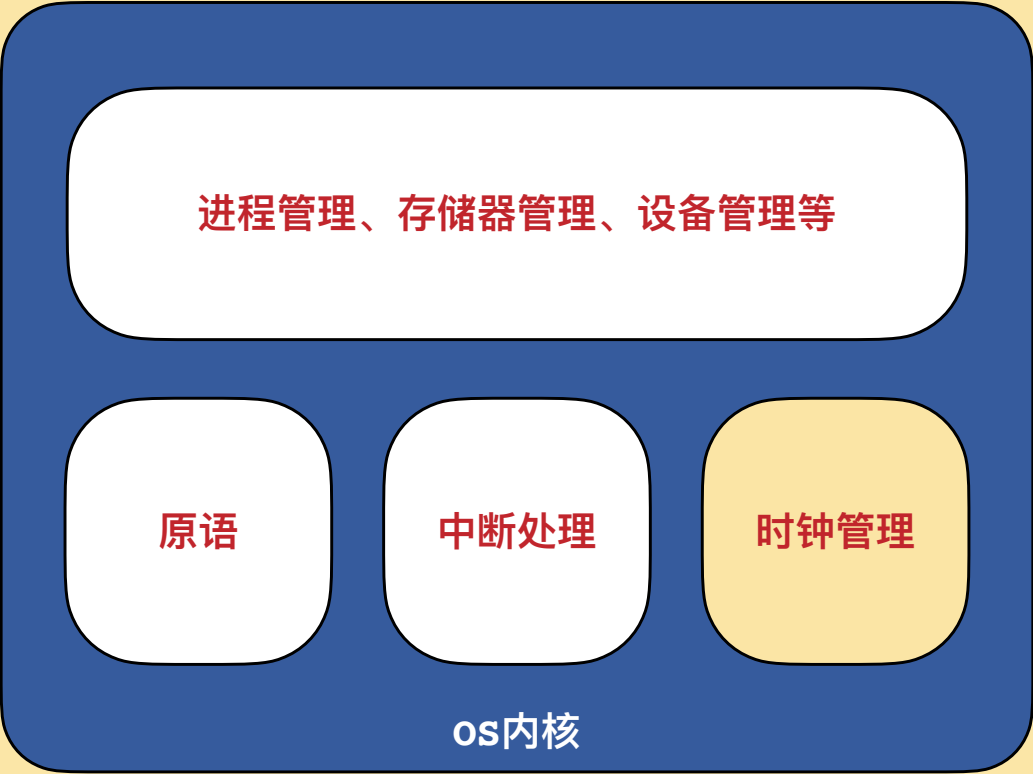
系统调用、键盘命令输入、进程调度、设备驱动等很多重要的活动都依赖于中断



1. 操作系统的运行环境

- os引导
- os内核**
- cpu的双重工作模式
- 特权指令
- 系统调用
- 中断和异常处理

os中与硬件紧密相关的模块，负责管理系统的各种资源



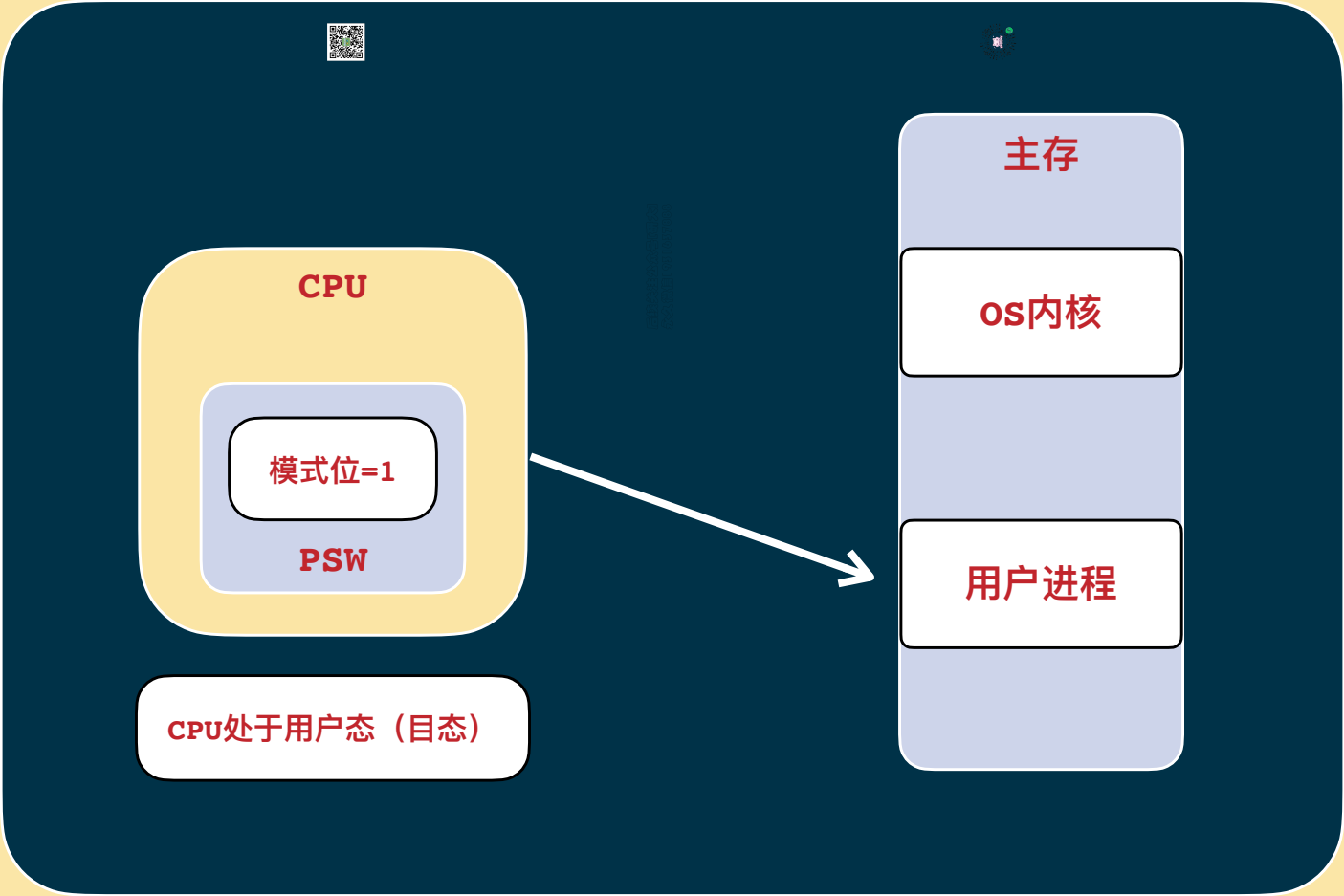
时钟管理

os用来处理和控制时间和时间相关函数的机制



1. 操作系统的运行环境

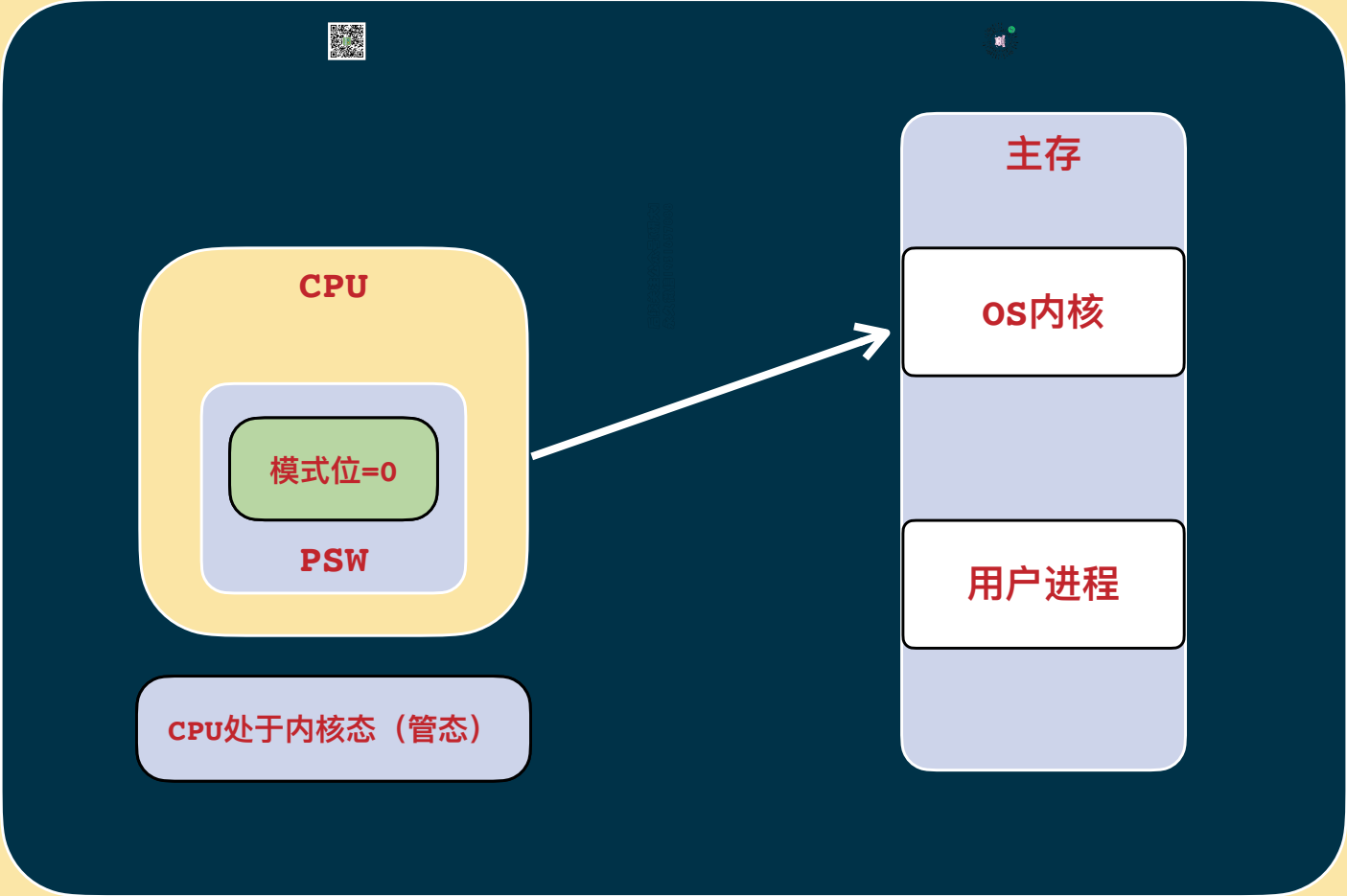
- os引导
- os内核
- CPU的双重工作模式
- 特权指令
- 系统调用
- 中断和异常处理





1. 操作系统的运行环境

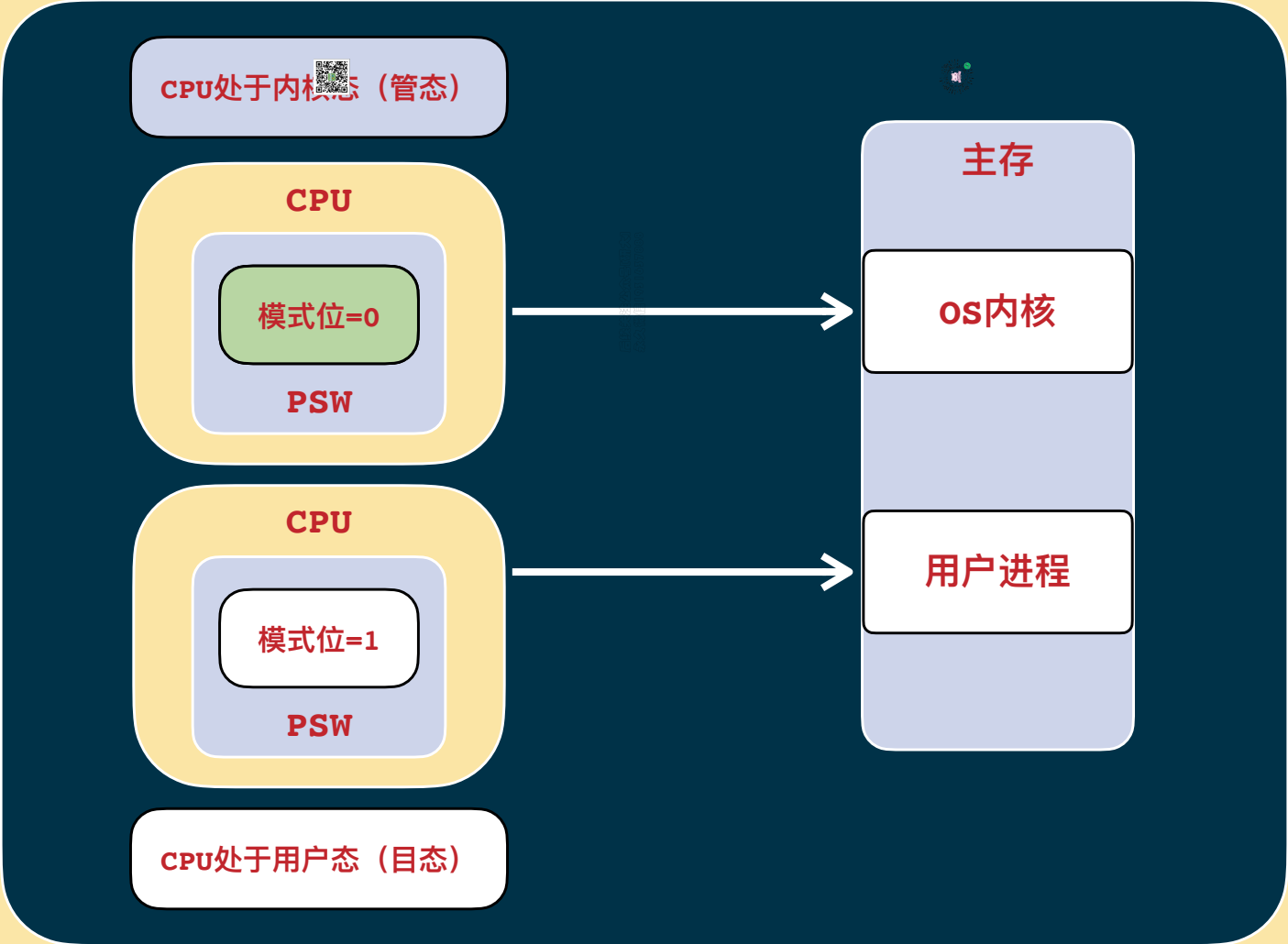
- os引导
- os内核
- CPU的双重工作模式
- 特权指令
- 系统调用
- 中断和异常处理





1. 操作系统的运行环境

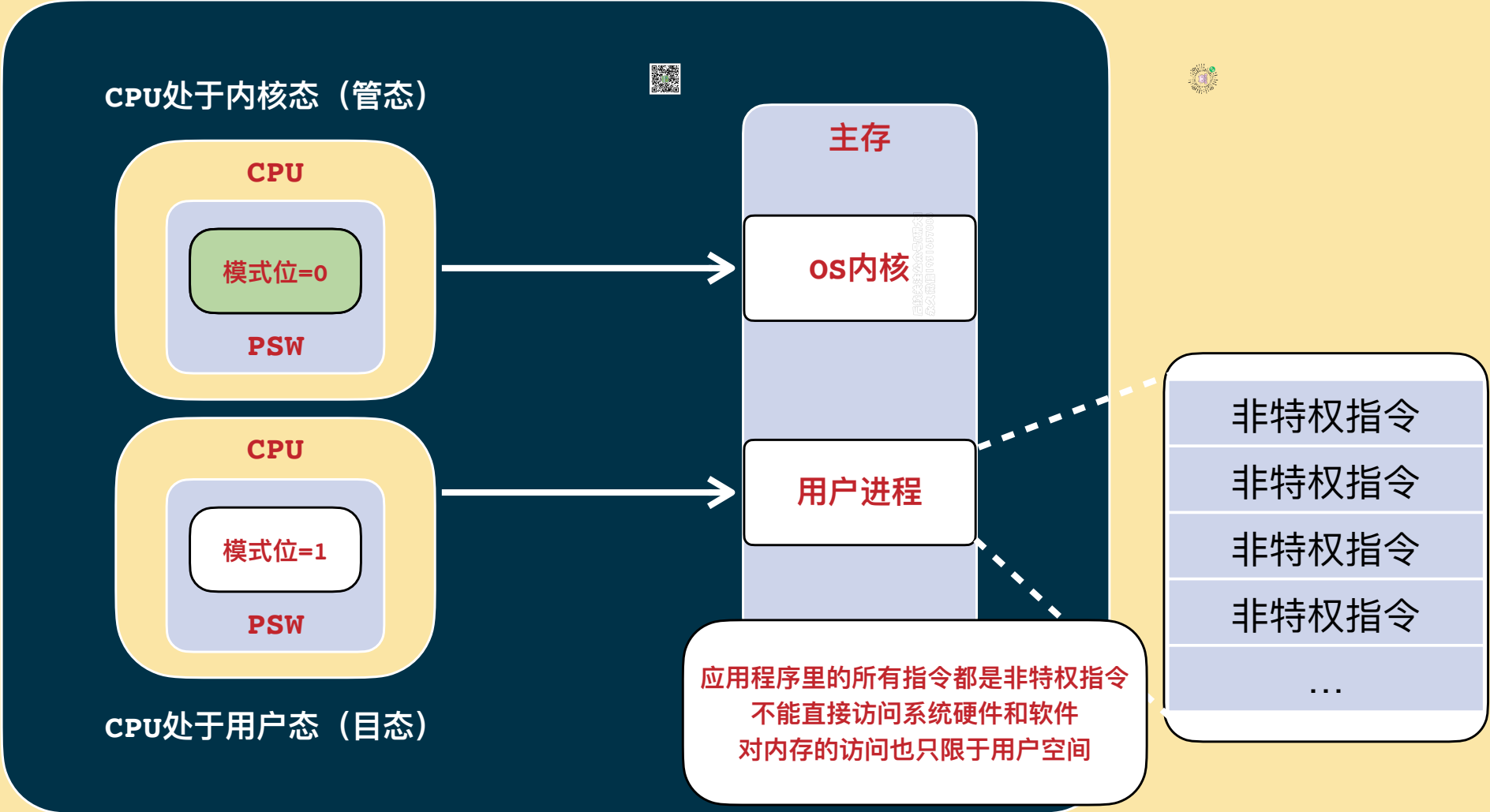
- os引导
- os内核
- CPU的双重工作模式
- 特权指令
- 系统调用
- 中断和异常处理





1. 操作系统的运行环境

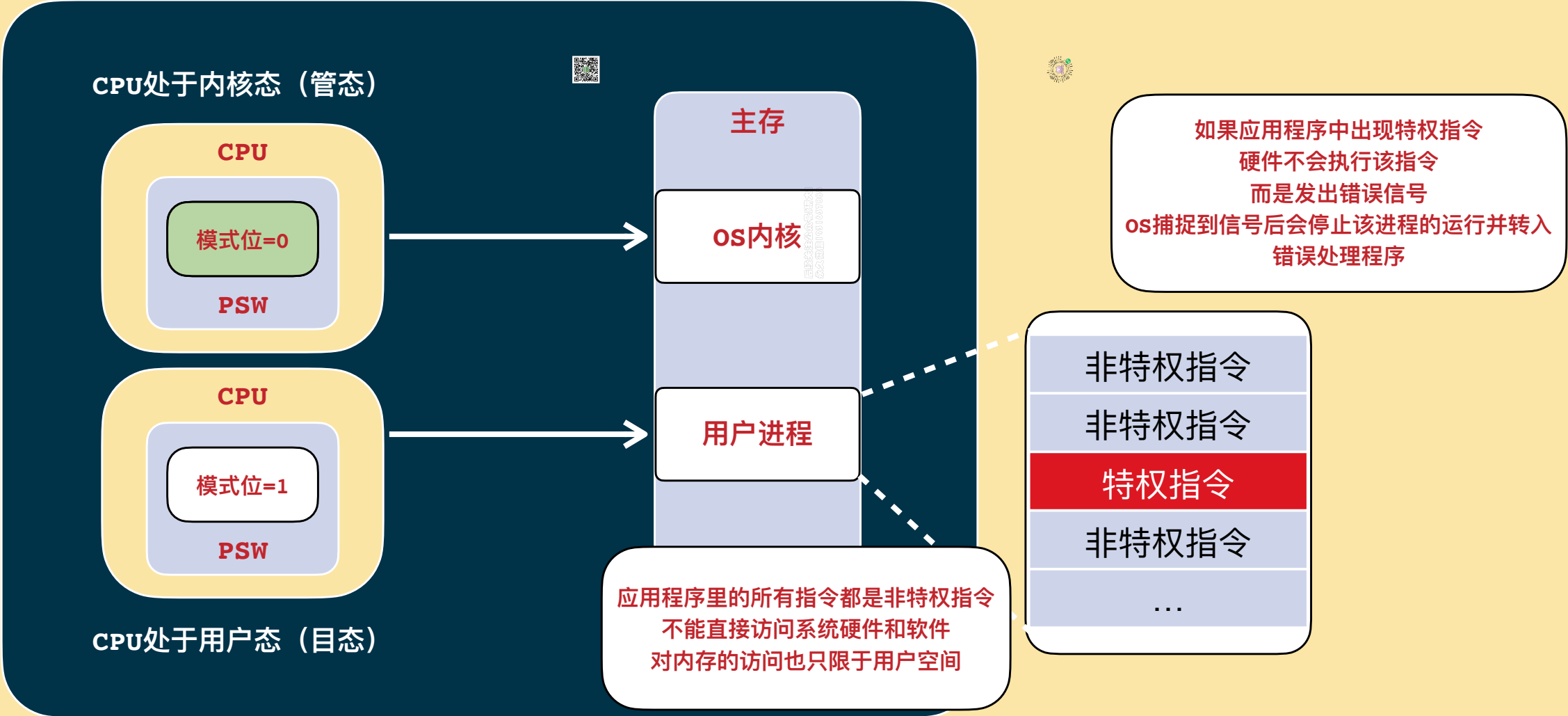
- os引导
- os内核
- CPU的双重工作模式
- 特权指令
- 系统调用
- 中断和异常处理





1. 操作系统的运行环境

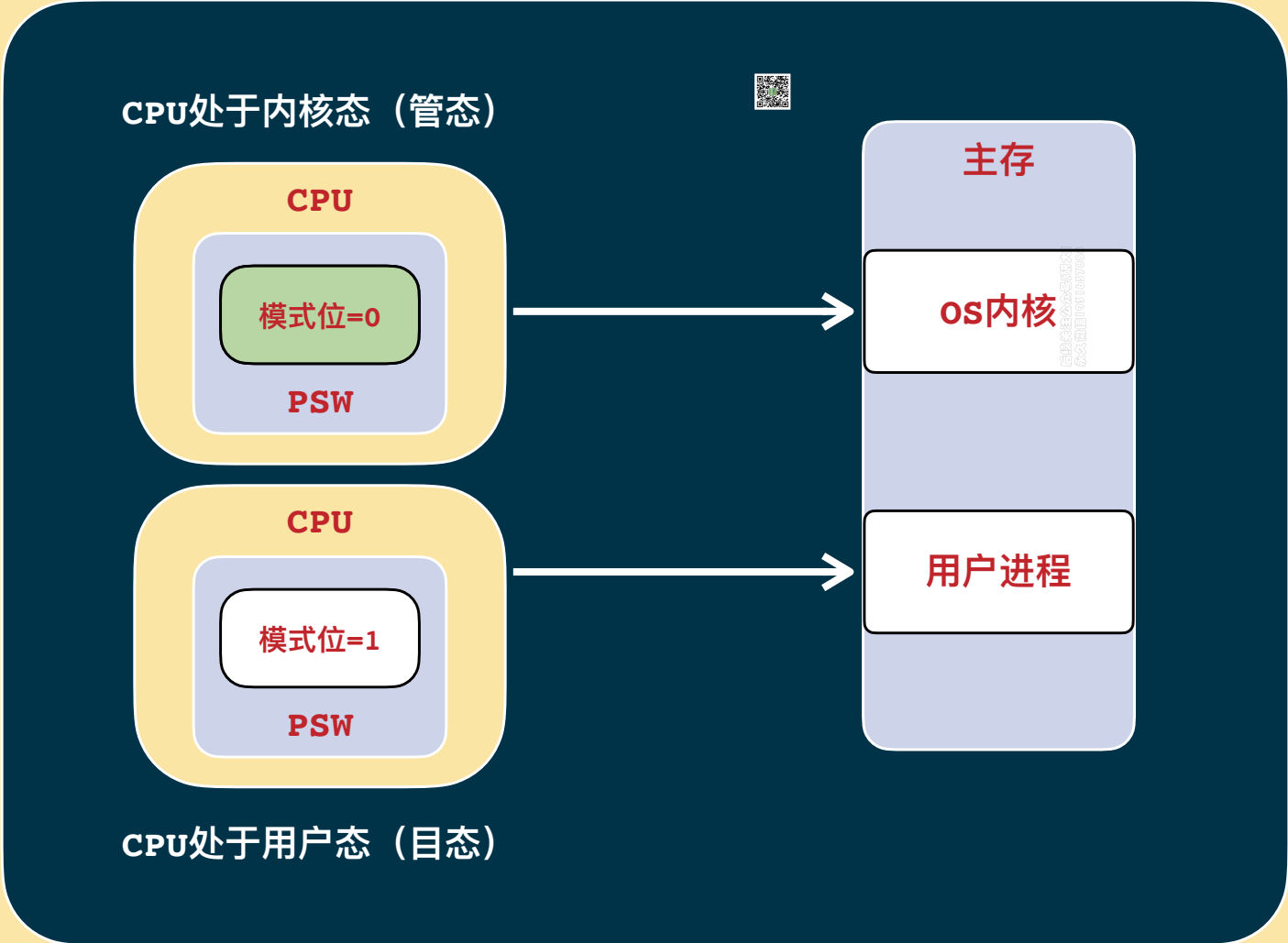
- os引导
- os内核
- CPU的双重工作模式
- 特权指令**
- 系统调用
- 中断和异常处理





1. 操作系统的运行环境

- os引导
- os内核
- CPU的双重工作模式
- 特权指令**
- 系统调用
- 中断和异常处理



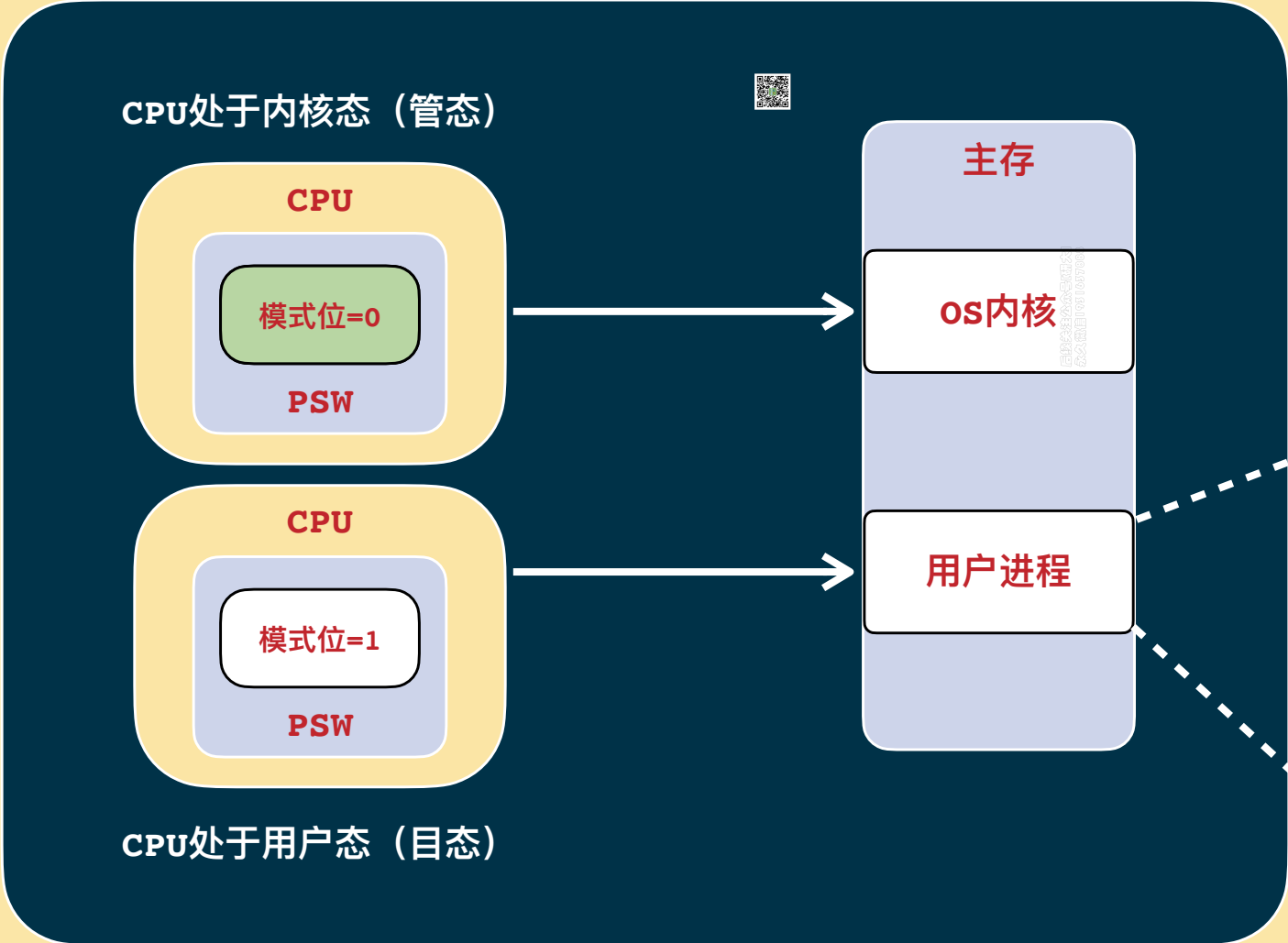
特权指令

特权指令是CPU在内核态才能运行的指令
它对内存的访问范围基本不受限制



1. 操作系统的运行环境

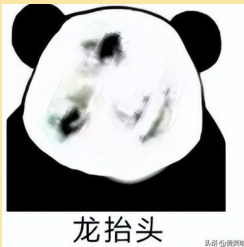
- os引导
- os内核
- CPU的双重工作模式
- 特权指令
- 系统调用**
- 中断和异常处理



用户程序想要使用os的服务的时候
怎么办?



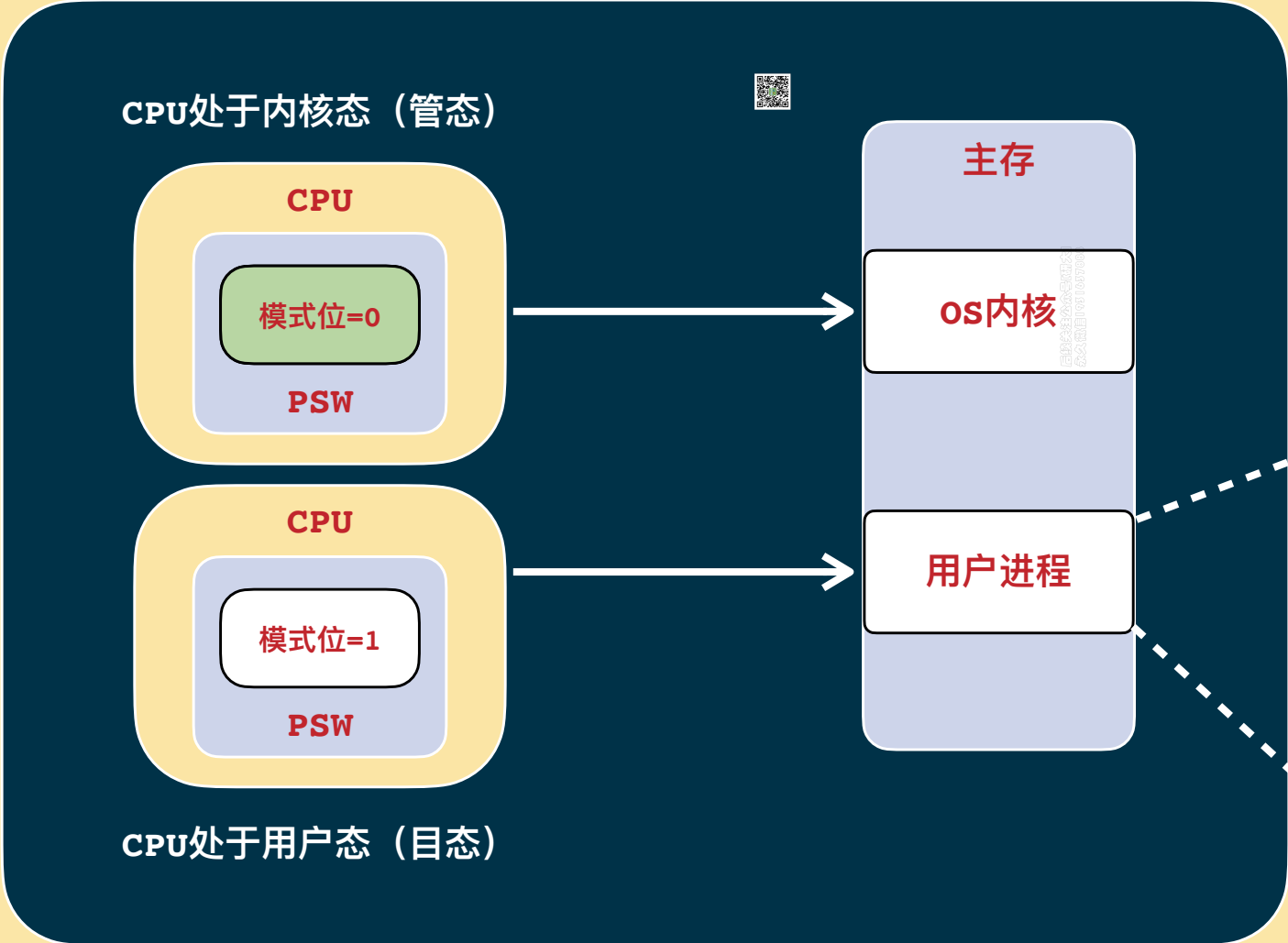
内核:





1. 操作系统的运行环境

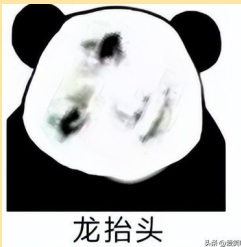
- os引导
- os内核
- CPU的双重工作模式
- 特权指令
- 系统调用
- 中断和异常处理



用户程序想要使用os的服务的时候
怎么办?



内核:



用户进程可以通过系统调用来使用os的文件管理服务

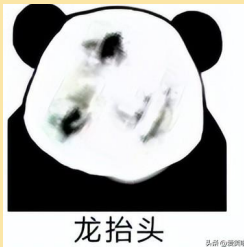


1. 操作系统的运行环境

- os引导
- os内核
- CPU的双重工作模式
- 特权指令
- 系统调用**
- 中断和异常处理

用户程序想要使用os的服务的时候
怎么办？

内核：



龙抬头

用户进程可以通过系统调用来使用os的文件管理服务

文件操作：

- `open()`：打开一个文件。
- `read()`：从文件中读取数据。
- `write()`：向文件写入数据。
- `close()`：关闭一个打开的文件描述符。
- `lseek()`：移动文件的读写位置。
- `unlink()`：删除一个文件的目录项和减少文件的链接数。
- `stat()`, `fstat()`：获取文件的状态信息。

进程间通信：

- `pipe()`：创建一个管道，用于进程间的通信。
- `shmget()`, `shmat()`, `shmctl()`：进行共享内存的操作。
- `semget()`, `semop()`, `semctl()`：信号量的操作。
- `msgget()`, `msgsnd()`, `msgrcv()`：消息队列的操作。

进程控制：

- `fork()`：复制当前进程，创建一个新的进程。
- `exit()`：结束当前进程。
- `execve()`：执行一个新程序，替换当前进程的映像。
- `wait()`, `waitpid()`：等待子进程结束，并获取子进程的终止状态。
- `kill()`：向任何进程或进程组发送信号。

非特权指令

非特权指令

`read(fd,buf,count)`

非特权指令

...



1. 操作系统的运行环境

- os引导
- os内核
- CPU的双重工作模式
- 特权指令
- 系统调用**
- 中断和异常处理



系统调用

用户代码中的read() 只是一个库函数
并非真正的系统调用

read(fd,buf,count)

库函数准备所有必要的参数
设置寄存器

这个指令不会显式地出现在用户代码中
用户会以为是自己的read() 引发了系统调用
库函数偷偷地执行一个**软中断或陷阱指令**
发起真正的系统调用

int 0x80/syscall





1. 操作系统的运行环境

- os引导
- os内核
- CPU的双重工作模式
- 特权指令
- 系统调用
- 中断和异常处理



用户代码中的read() 只是一个库函数
并非真正的系统调用

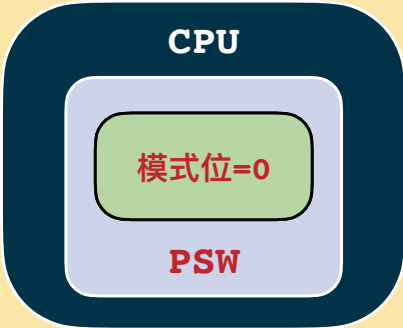
read(fd,buf,count)

库函数准备所有必要的参数
设置寄存器

这个指令不会显式地出现在用户代码中
用户会以为是自己的read() 引发了系统调用
库函数偷偷地执行一个软中断或陷阱指令
发起真正的系统调用

int 0x80/syscall

执行软中断/陷阱指令
可以引发自陷异常
导致CPU从用户态陷入到内核态





1. 操作系统的运行环境



误区



“自陷/陷入”并不是指一条指令
让CPU陷入到内核态的是软中断指令/陷阱指令
而CPU陷入到内核态的这个动作属于自陷/陷入，它是一种现象
自陷/陷入这种异常指的是主动设置一起异常事件，让CPU进入异常处理
正好符合系统调用的思路
只有CPU进入内核态，才能执行os的代码，应用才能请求os的服务

用户代码中的read() 只是一个库函数
并非真正的系统调用

read(fd,buf,count)

库函数准备所有必要的参数
设置寄存器

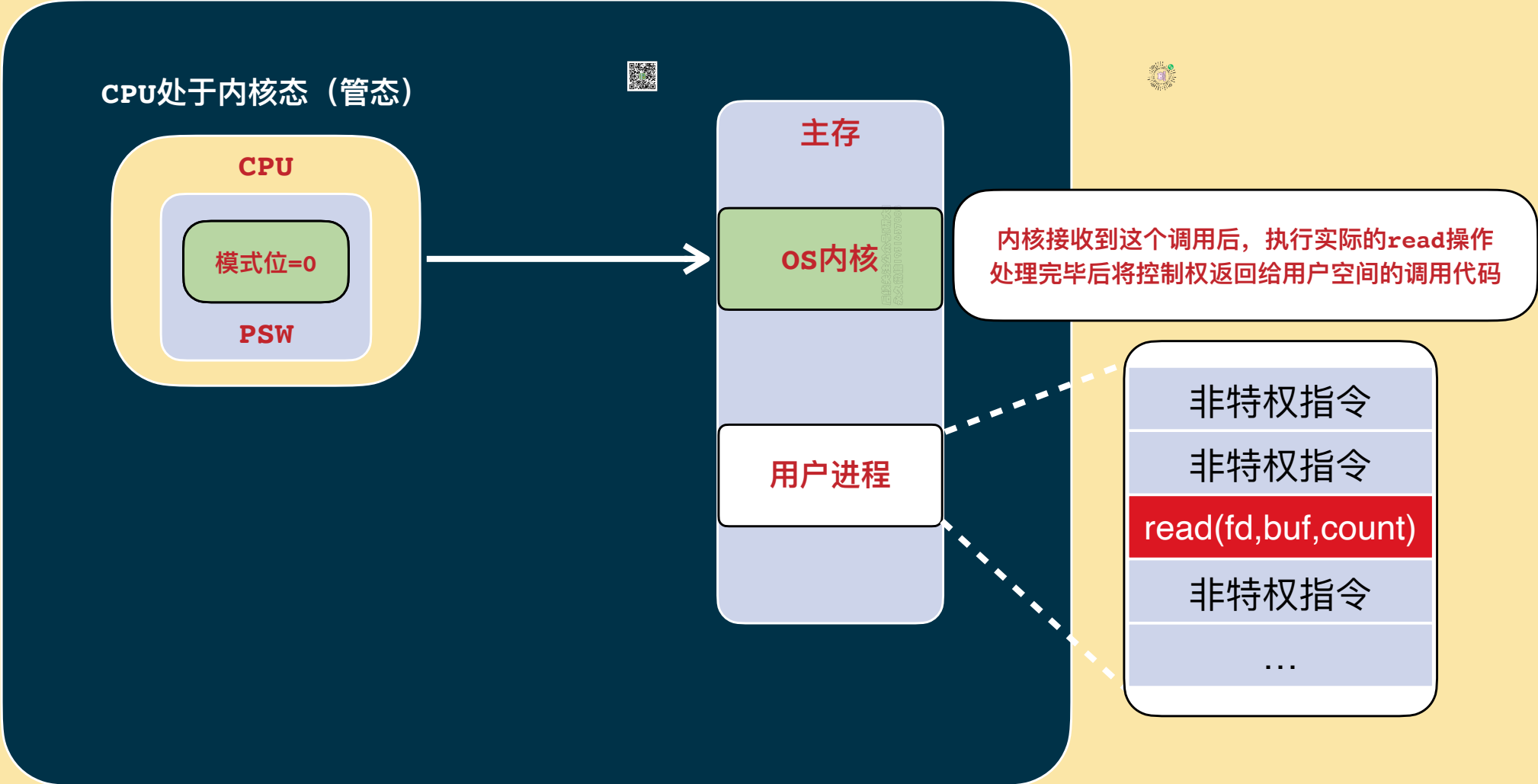
这个指令不会显式地出现在用户代码中
用户会以为是自己的read() 引发了系统调用
库函数偷偷地执行一个软中断或陷阱指令
发起真正的系统调用

int 0x80/syscall



1. 操作系统的运行环境

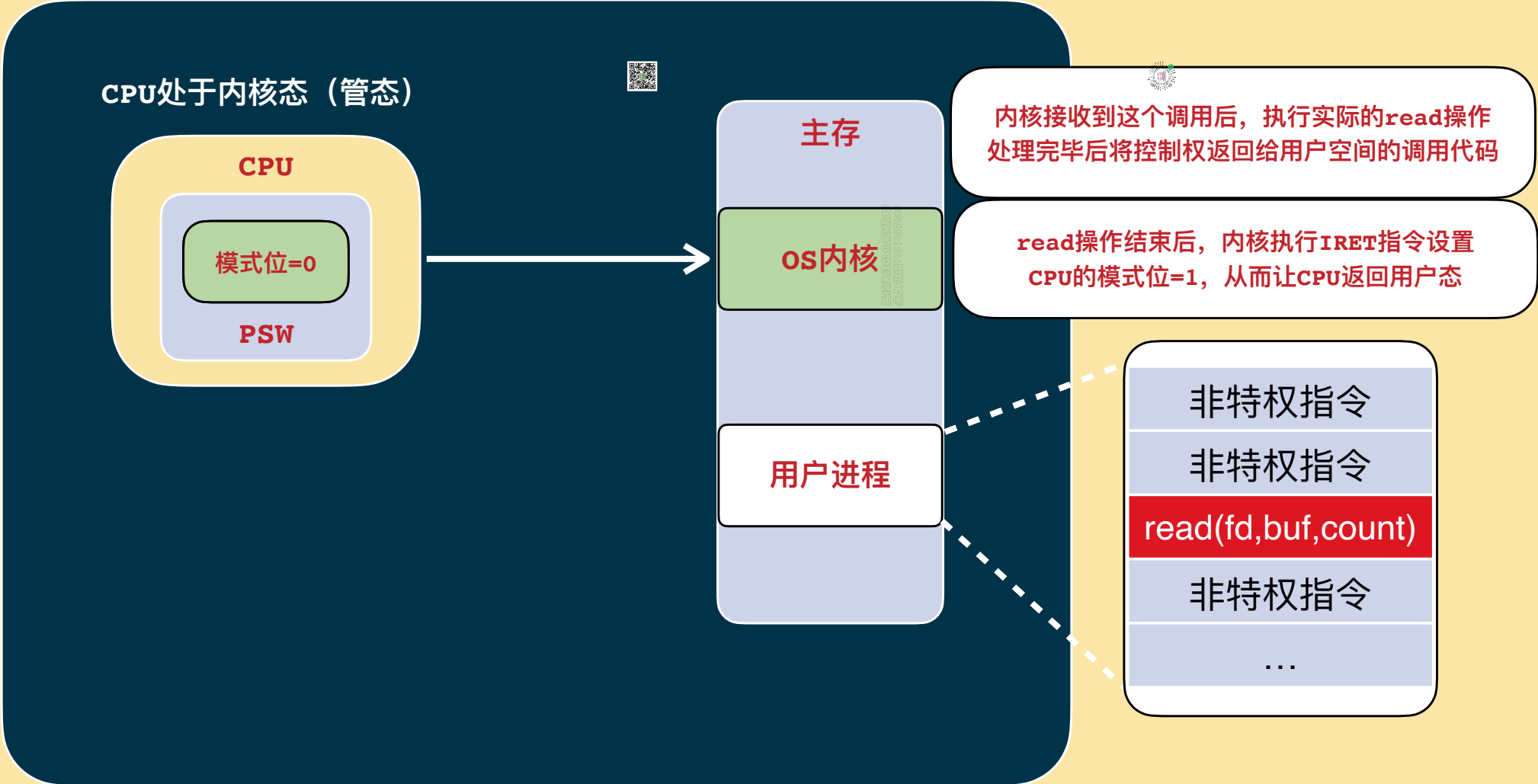
- os引导
- os内核
- CPU的双重工作模式
- 特权指令
- 系统调用
- 中断和异常处理





1. 操作系统的运行环境

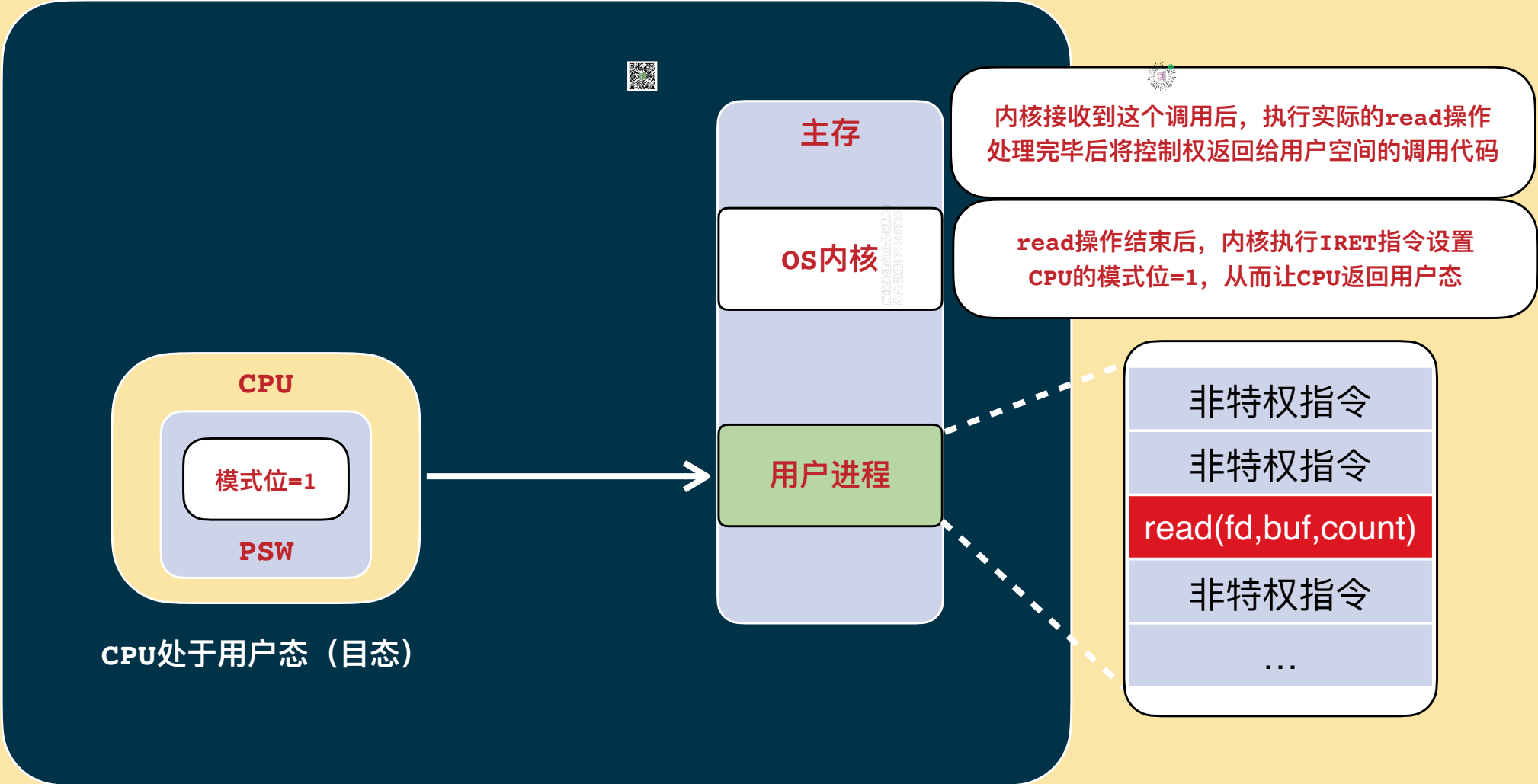
- os引导
- os内核
- CPU的双重工作模式
- 特权指令
- 系统调用
- 中断和异常处理





1. 操作系统的运行环境

- os引导
- os内核
- CPU的双重工作模式
- 特权指令
- 系统调用
- 中断和异常处理





1. 操作系统的运行环境

os引导

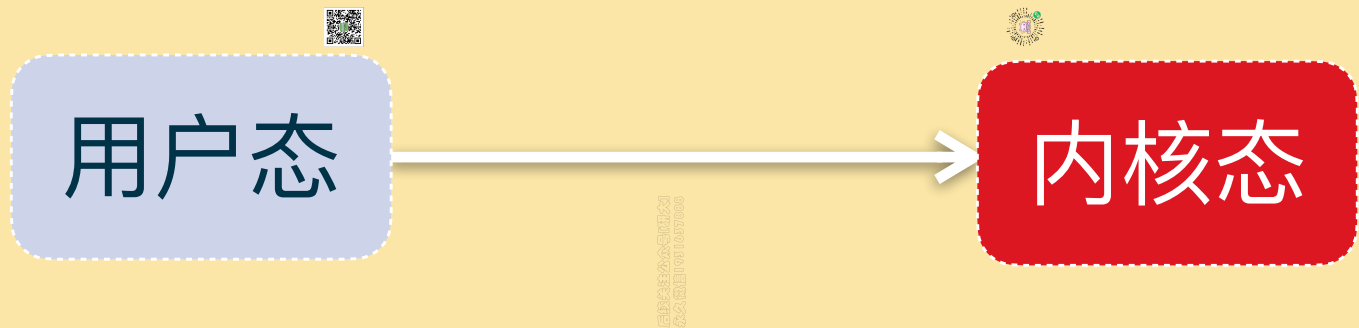
os内核

CPU的双重工作模式

特权指令

系统调用

中断和异常处理



1. 用户程序发起系统调用

2. 发生中断，必须陷入内核态由os内核处理中断

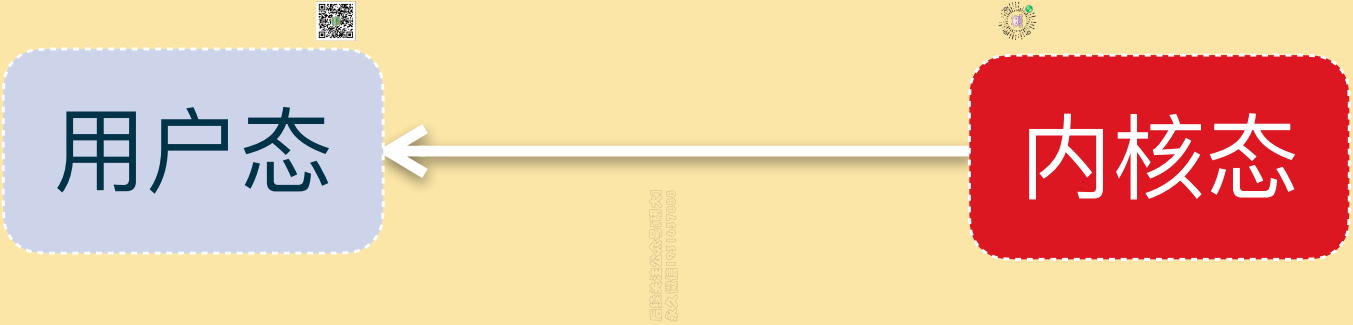
3. 发生异常，必须陷入内核态由os内核处理中断

4. 用户程序试图执行特权指令



1. 操作系统的运行环境

- os引导
- os内核
- CPU的双重工作模式
- 特权指令
- 系统调用**
- 中断和异常处理

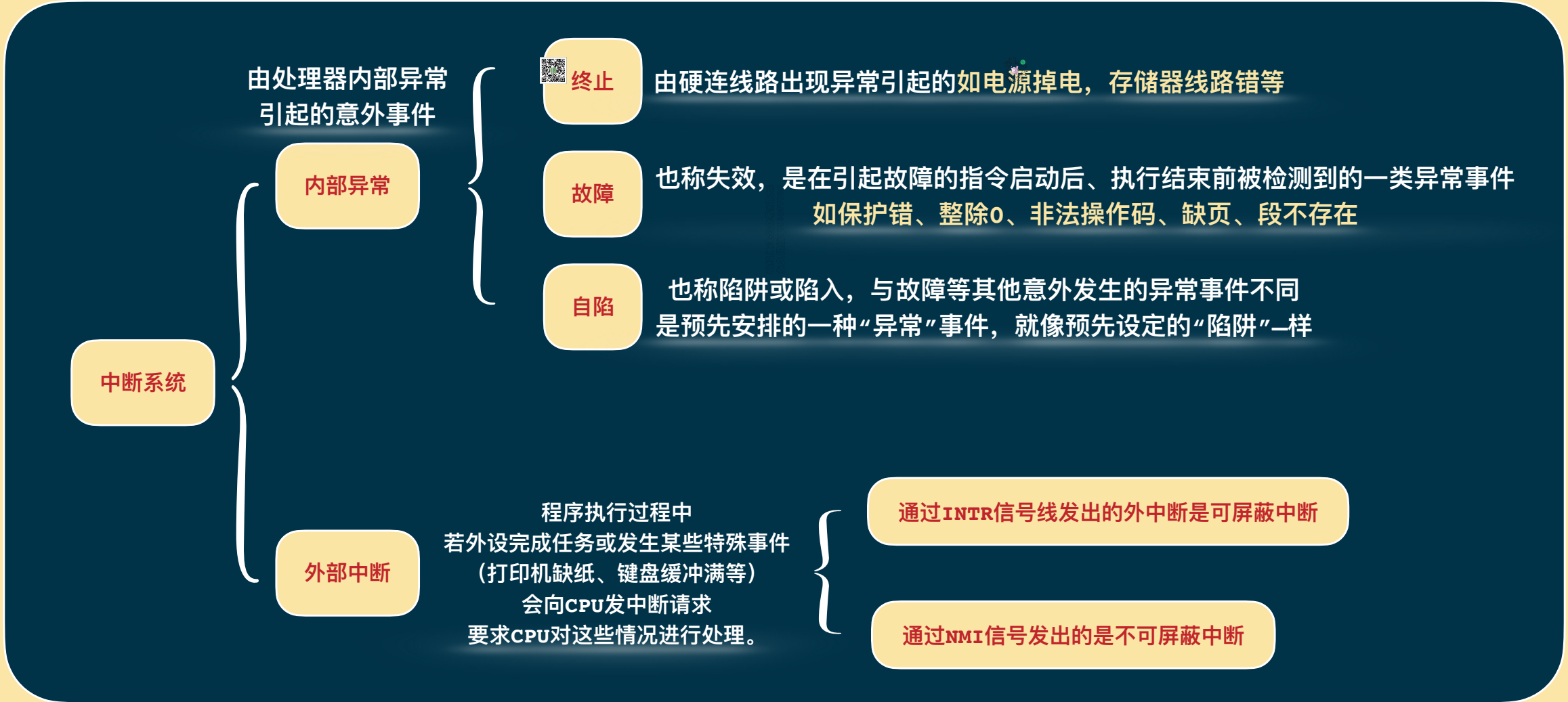


设置程序状态字寄存器PSW中的CPU模式位，将其置1



1. 操作系统的运行环境

- os引导
- os内核
- CPU的双重工作模式
- 特权指令
- 系统调用
- 中断和异常处理





1. 操作系统的运行环境

OS引导

OS内核

CPU的双重工作模式

特权指令

系统调用

中断和异常处理

异常处理过程

异常触发：当程序执行中发生异常，CPU会自动检测并触发异常

保存上下文：与中断处理相似，CPU会保存当前的执行状态，包括程序计数器和寄存器值

执行异常处理程序：CPU查找异常向量表，根据异常类型跳转到相应异常处理程序处理

处理结果：根据异常性质，处理程序可能会修正错误并允许程序继续执行，或终止程序返回错误信息

恢复或终止：如果异常可恢复，CPU将恢复之前的程序状态并继续执行；若不可恢复，终止程序并进行错误处理



1. 操作系统的运行环境

OS引导

OS内核

CPU的双重工作模式

特权指令

系统调用

中断和异常处理

中断处理过程

触发中断：中断指由外部设备（键盘、鼠标等）发起，通常是设备完成任务或需要注意时触发

中断信号接收：CPU会在当前执行的指令完成后检测中断，暂停当前进程的执行

保存上下文：CPU会保存当前任务的状态，包括程序计数器和其他寄存器值（现场信息），以便中断处理完成后能恢复运行

执行中断服务程序：CPU根据中断向量表找到相应中断处理程序并执行

恢复上下文并返回：中断服务程序完成后，CPU恢复之前保存的程序状态，继续执行被中断程序



1. 操作系统的运行环境

OS引导

OS内核

CPU的双重工作模式

特权指令

系统调用

中断和异常处理

中断处理过程

异常处理过程

触发中断：中断指由外部设备（键盘、鼠标等）发起，通常是设备完成任务或需要注意时触发

异常触发：当程序执行中发生异常，CPU会自动检测并触发异常

中断信号接收：CPU会在当前执行的指令完成后检测中断，暂停当前进程的执行

保存上下文：与中断处理相似，CPU会保存当前的执行状态，包括程序计数器和寄存器值

保存上下文：CPU会保存当前任务的状态，包括程序计数器和其他寄存器值（现场信息），以便中断处理完成后能恢复运行

执行异常处理程序：CPU查找异常向量表，根据异常类型跳转到相应异常处理程序处理

执行中断服务程序：CPU根据中断向量表找到相应中断处理程序并执行

处理结果：根据异常性质，处理程序可能会修正错误并允许程序继续执行，或终止程序返回错误信息

恢复上下文并返回：中断服务程序完成后，CPU恢复之前保存的程序状态，继续执行被中断程序

恢复或终止：如果异常可恢复，CPU将恢复之前的程序状态并继续执行；若不可恢复，终止程序并进行错误处理



真题训练

*此模块题目取自真题及与真题风格类似的模拟题

1. 【2013统考真题】计算机开机后，操作系统最终被加载到()

A. BIOS

B. ROM

C. EPROM

D. RAM



真题训练

*此模块题目取自真题及与真题风格类似的模拟题

1. 【2013统考真题】计算机开机后，操作系统最终被加载到()

A. BIOS

B. ROM

C. EPROM

D. RAM



真题训练

*此模块题目取自真题及与真题风格类似的模拟题

2. 【2012 统考真题】下列选项中，不可能在用户态发生的事件是()

A. 系统调用

B. 外中断

C. 进程切换

D. 缺页



真题训练

*此模块题目取自真题及与真题风格类似的模拟题

2. 【2012 统考真题】下列选项中，不可能在用户态发生的事件是()

A. 系统调用

B. 外中断

C. 进程切换

D. 缺页



真题训练

*此模块题目取自真题及与真题风格类似的模拟题

3. 【2013统考真题】下列选项中，会导致用户进程从用户态切换到内核态的操作是（）。

I. 整数除以零  II. `sin()`函数调用 III. `read`系统调用

A. I、II

B. I、III

C. II、III

D. I、II和III



真题训练

*此模块题目取自真题及与真题风格类似的模拟题

3. 【2013统考真题】下列选项中，会导致用户进程从用户态切换到内核态的操作是（）。

I. 整数除以零  II. `sin()` 函数调用 III. `read` 系统调用

A. I、II

B. I、III

C. II、III

D. I、II和III



真题训练

*此模块题目取自真题及与真题风格类似的模拟题

4. 【2024统考真题】下列关于中断、异常和系统调用的叙述中，错误的是（）。



- A. 中断或异常发生时，CPU处于内核态
- B. 每个系统调用都有对应的内核服务例程
- C. 中断处理程序开始执行时，CPU 处于内核态
- D. 系统添加新类型设备时，需要注册相应的中断服务例程



真题训练

*此模块题目取自真题及与真题风格类似的模拟题

4. 【2024统考真题】下列关于中断、异常和系统调用的叙述中，错误的是（）。



- A. 中断或异常发生时，CPU处于内核态**
- B. 每个系统调用都有对应的内核服务例程
- C. 中断处理程序开始执行时，CPU 处于内核态
- D. 系统添加新类型设备时，需要注册相应的中断服务例程